

Tema 1: Introducción a los sistemas operativos.

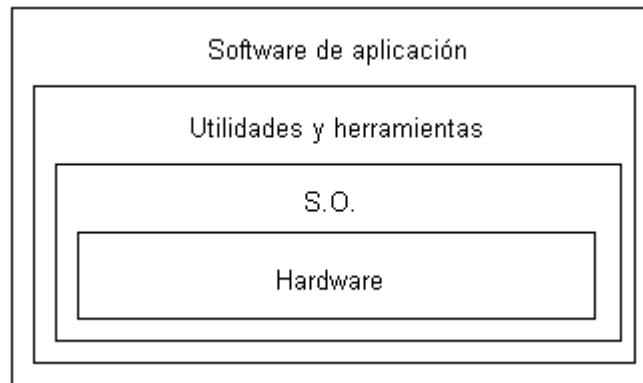
1.- ¿Qué es un S.O.?

Son un conjunto de programas que actúan como interface ó comunicación hombre – máquina (hardware), ofreciendo un entorno necesario para que el usuario pueda ejecutar sus programas, pero que a la vez limita la capacidad de la máquina, pues no podemos utilizar todos los recursos.

El S.O. tiene principalmente dos objetivos:

1. Facilitar el uso del sistema informático (el conjunto del hardware y software).
2. Empleo del hardware de forma eficiente, con menor complejidad.

También podemos definir el S.O. como una parte importante de un sistema informático, pudiendo dividir el software de este en varias capas, de la siguiente forma:



Las utilidades y las herramientas sirven para dar soporte a otros programas y el software de aplicación es el que soluciona problemas concretos y que está orientado a usuarios.

CAPAS:

- ☛ *S.O.*: Mantiene un entorno necesario para la ejecución de los programas de usuario.
- ☛ *Herramientas*: Son programas que nos ayudan. Proporcionan comodidad para el usuario en el uso de programas.
Ejemplos:
 - Sistemas de gestión de bases de datos
 - Intérpretes de ordenes (Shell): interface construido sobre el S.O., tiene ciertos interpretes sobre el S.O.
 - Depuradores (Debugger): ayuda a corregir ó depurar programas.
 - Programas de tratamiento de ficheros.
 - Editores de texto (no procesadores): solo sirven para escribir.
 - Comunicación: compiladores.
 - Traductores: interpretes.
- ☛ *Software de aplicación*: Software construido para resolver problemas concretos (Ejemplo: Word Perfect).
- ☛ *Software de sistema*: Software construido para la ayuda de otros problemas, no para la ayuda del usuario.

Funciones del S.O.:

1. Ofrecer al usuario una máquina virtual que es más fácil de programar que el hardware.
2. Funciona como gestor de recursos, buscando interactuar con la máquina y usar la C.P.U. sin perder tiempo.

2.- Evolución histórica.ETAPA 0:

Sólo existe hardware, no hay S.O. El programador escribía el programa y lo ejecutaba desde la consola del operador. El programa se cargaba manualmente en memoria, seguidamente se cargaba la dirección de inicio, en ese momento comenzaba la ejecución del programa, el programador seguía la misma mediante los pilotos de la consola. Podía corregir errores sobre la marcha, es decir, desde la misma consola. La salida se imprimía, se grababa en cinta ó se perforaba en una tarjeta.

Pero en esta etapa había un grave problema y era el método interactivo de reserva de tiempo de la C.P.U., ya que cada persona reservaba tiempo en una hoja de solicitud y este se calculaba a ojo. Si te faltaba tiempo, tenías que volver a continuar en otro momento, mientras que si te sobraba, la C.P.U. estaba inactiva. Por tanto el tiempo de preparación para la resolución de problemas era muy alto y como consecuencia, existía desperdicio de la C.P.U. No había S.O., sólo máquina trabajando b bajo uso de la C.P.U.

ETAPA 1:

Aparecen las cintas magnéticas. Tendremos 2 ordenadores, uno que saca cintas magnéticas (satélite) y otro (central). El central lee más rápido.

En esta etapa, lo que se intentó solucionar sobre todo era el desperdicio de tiempo de la C.P.U., ya que cada trabajo implicaba un tiempo de preparación considerable, por ejemplo, si queríamos ejecutar un programa en Fortram los pasos a seguir eran los siguientes:

- ☞ Cargar la cinta con el compilador de Fortram.
- ☞ Ejecutar el compilador de Fortram.
- ☞ Descargar la cinta del compilador.
- ☞ Cargar la cinta del ensamblador, ya que el compilador de Fortram generaba una salida en lenguaje ensamblador que tenía que ser ensamblada.
- ☞ Ejecución del ensamblador.
- ☞ Descarga de la cinta del ensamblador.
- ☞ Carga del programa objeto.
- ☞ Ejecución del programa objeto.

Y si tenías la desgracia de que se produjese un error en uno de los pasos, quizás tuvieras que empezar de nuevo.

Para solucionar el desperdicio de tiempo se encontraron 2 soluciones:

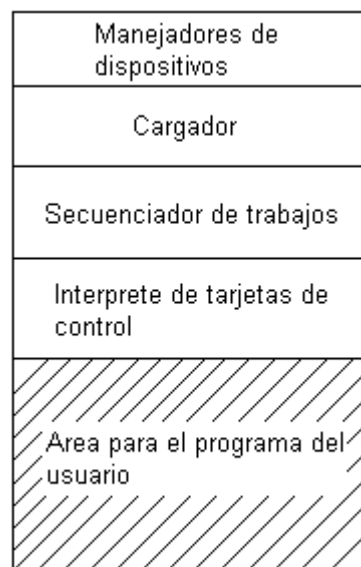
1. Se contrato a un operador de ordenadores profesional, así el programador no operaba la máquina. Una vez finalizado un trabajo, el operador podía iniciar el siguiente, por lo que había dos personas encargadas del ordenador.

2. Reducción del tiempo de preparación, los trabajos con necesidades similares, se reunieron por lotes (ficheros BATCH), y se ejecutaban en el ordenador como un grupo.

Una vez solucionado relativamente el desperdicio de tiempo, surgía un nuevo problema cuando un trabajo se detenía, ya que los operadores tenían que darse cuenta observando la consola, determinar porque se había detenido la ejecución del programa, realizar un volcado si era necesario, cargar la lectora de tarjetas ó cintas, etc., lo que hacía que la C.P.U. permaneciera inactiva. Para eliminar este tiempo de inactividad se introdujo el secuenciamiento automático de trabajos y con él se crearon los primeros y rudimentarios S.O..

Lo que se necesitaba era un procedimiento para la transferencia automática del control de un trabajo al siguiente. Así se creó un pequeño programa llamado Monitor Residente (ya que reside siempre en memoria). Al arrancar el ordenador el control estaba en el monitor residente, quien lo transfería al programa, hasta que se terminaba la ejecución del mismo, momento en el que se devolvía el control al monitor residente que continuaba con el programa siguiente. Era una forma de ejecutar secuencialmente todos los programas.

Para guiar al monitor residente de que programas debía ejecutar, se introdujeron las tarjetas de control. Un ejemplo de ello es :



Dándose cuenta que la C.P.U. quedaba inactiva el 93.7% del tiempo por culpa de los dispositivos de E/S (tarjetas perforadas), se medio solucionó el problema pasando primero la información de las tarjetas a cintas magnéticas: se pasaba la información de la tarjeta a la cinta, seguidamente se ejecutaba en el ordenador, la salida iba a otra cinta y por último si se creía conveniente se pasaba de nuevo a tarjeta. Por tanto las lectoras de tarjetas y las impresoras de línea eran operadas OFF – LINE, i.e., no por el ordenador principal. Para ello se necesitaba un pequeño ordenador que se dedicara a la tarea de copiar desde cinta ó en cinta. A éste se le llamaba Sistema Satélite y a la técnica de realizar la lectura desde tarjetas de manera adelantada se le llama Spooling.

ETAPA 2:

Aparece el concepto de interrupciones y controladores. Los controladores solapan las operaciones de E/S. Se trata de acelerar al máximo el tiempo de C.P.U..

Aparece el concepto de Multiprogramación que es un sistema para mantener ocupada el mayor tiempo posible la C.P.U., ya que existen varios procesos simultáneamente en trabajos de un grupo de ellos, si por algún motivo (especialmente operaciones E/S o término del tiempo asignado en los sistemas más modernos) hay que esperar, el S.O. cambiará a otro trabajo y lo ejecutará. Eso ocurrirá con todos los trabajos. Exige planificar los trabajos.

Multiproceso: son varios los procesos que pueden estar activos a la vez, pero solo los datos e instrucciones de uno de ellos está activo en Memoria principal (un ordenador que esté realizando varios trabajos a la vez).

Multiusuario: varios procesos pueden estar conectados a través de terminales a una computadora (como en la Escuela).

Sistemas de tiempo compartido: Los primeros sistemas Batch eran una agrupación de trabajos similares, en los que había que esperar que acabase un trabajo para que empezase otro. Los sistemas basados en cintas o tarjetas solo permitían el acceso secuencial a los programas y datos y solo podía utilizarse una aplicación a la vez. Cuando se dispuso del almacenamiento en disco, i.e. acceso aleatorio, un S.O. Batch lee una serie de tareas diferentes, normalmente, cuando el trabajo se ha concluido imprime su salida. La característica definitiva de un sistema Batch es la falta de interacción entre el usuario y el trabajo, ya que este podía salir horas o incluso días más tarde. Por tanto estos sistemas tienen una serie de inconvenientes que son:

- ✍ El usuario no podía interaccionar en los procesos mientras se estaba ejecutando.
- ✍ La depuración de los programas se realizaba estáticamente, a partir de vocablos puntuales.

Un sistema informático interactivo o Hands – on, facilita la comunicación On – Line entre el usuario y el sistema. Los Sistemas de Tiempo Compartido son el resultado de la tentativa de obtener un funcionamiento interactivo de un sistema informático. Utiliza la planificación de la C.P.U. y la multiprogramación para dotar al usuario de una pequeña parte del ordenador compartido. Podemos trabajar con el ordenador aunque haya otros en la red. Este sistema permite que muchos usuarios compartan simultáneamente el ordenador, compartiendo el tiempo con todos los programas que se están ejecutando, siendo el tiempo de C.P.U. o de cambio de programas muy corto y el usuario no lo nota, cree que está sólo trabajando con el ordenador.

Estos sistemas necesitan introducir sistemas de seguridad, algún mecanismo de ejecución concurrente de procesos y mecanismo de protección de memoria

Sistemas en Tiempo Real: Este se utiliza frecuentemente como dispositivo de control en una aplicación especializada, con sensores que proporcionan datos al ordenador y a la vez éste se encarga de ajustarlos correctamente. Tienen que garantizar un tiempo mínimo de respuesta. Normalmente es Monousuario y se construye para entornos donde puede haber muchos sucesos. La mayoría de los sucesos son externos al S.O. y debería realizarse en un tiempo breve. Se construye porque existe un proceso que se va a encargar de cada suceso que tiene una cierta prioridad, depende de la importancia o de lo crítico que sea, se le da mayor prioridad. Capturar información, capturar recursos, necesita un soporte de memoria para que pueda comunicarse.

ETAPA 3:

Aparecen los Sistemas de Propósito General, en los cuales pueden ejecutarse cualquier tipo de aplicación. Tenemos ordenadores que no sólo hagan una cosa.

También aparecen los Sistemas Interactivos.

Los Sistemas Multimodo, que son sistemas que aportan tanto el modo interactivo como el Batch. Debido a la complejidad de estos, se hace una estructuración de los S.O..

Aparece el UNÍX.

ETAPA 4:

En esta etapa se desarrollan los microprocesadores y con ellos el ordenador personal (P.C.). Este ordenador personal monousuario necesitaba un S.O. y aparece el MS – DOS.

Aparecen los Sistemas Distribuidos (las redes comunes).

Importancia de los Sistemas de Gestión de Bases de Datos.

3.- Distintos aspectos de los S.O.

Se va a determinar que requerimiento funcional y estructural de seguridad se va a requerir respecto a los siguientes puntos:

1. Respecto al modo de trabajo del usuario:

- a. ON – LINE: Interactivo. Estás trabajando sobre él.

Es un sistema de respuesta rápida, es decir, el usuario puede estar controlando el trabajo al mismo tiempo que se ejecuta. Pierde tiempo si el usuario está parado.

- b. OFF – LINE: No interactivo o Batch.

Mandas un proceso o trabajo. La C.P.U. no pierde tiempo si no le mandan un trabajo. Cuando no trabaja está parada. El usuario no puede seguir los pasos del programa o procesos, sólo cuando termina puede ver el resultado. Se ahorra tiempo de usuario. Tiende a aprovechar los recursos de la máquina.

2. Respecto al número de usuarios.

Según el número de usuarios se dividen en dos tipos:

- a. Monousuario.
b. Multiusuario (en este caso se debe usar sistema de protección de los datos).

3. Respecto al propósito.

Existen 2 tipos:

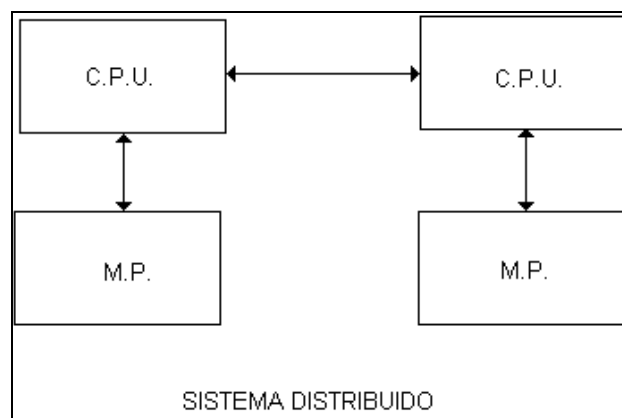
- a. S.O. de Propósito General: en este S.O. se puede ejecutar cualquier tipo de programa o aplicación (ejemplo: los PC's).
b. S.O. de Propósito Específico: sólo realizan una determinada tarea.

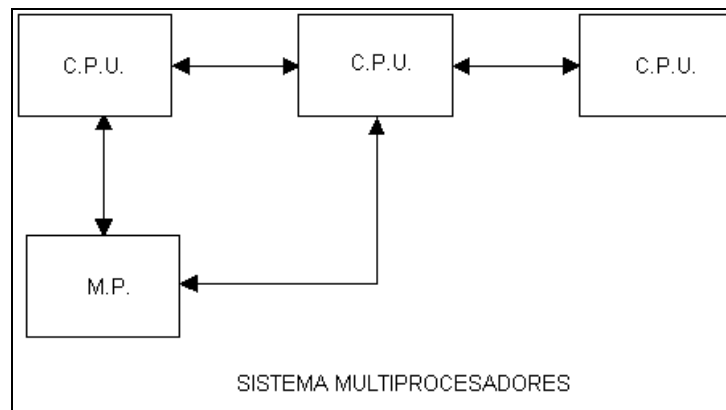
OBJETIVOS PRINCIPALES: Son el tiempo de acceso y el tiempo de respuesta donde deben ser muy rápidos, para que todos los usuarios tengan las mismas oportunidades.

SISTEMAS CON MÁS DE UNA C.P.U.:

Según la arquitectura existen 2 tipos:

- ☞ Sistemas multiprocesadores o paralelos: Tienen la memoria y el reloj común. Se ejecuta sobre el S.O. multiprocesador. Tiene ventajas como mayor potencia de computación y fiabilidad. No se conoce un modelo general de S.O. multiprocesador. Un usuario nos es consciente de que hay más de un usuario trabajando con el ordenador. Utiliza memoria mínima. El objetivo de estos sistemas es aumentar la productividad del sistema y para ello hay que obtener paralelismo de tareas, es decir, más de una C.P.U. destinada a una misma aplicación. Y se ejecutan varias tareas que se reparten entre las distintas C.P.U..
- ☞ Sistemas distribuidos: La memoria es local, es decir, no compartida. Los usuarios son conscientes de que hay varias máquinas. Hay 2 tipos de S.O._
 - Los de red: Los usuarios son conscientes de que hay varias máquinas. Cada nodo o cada máquina ejecuta su propio S.O. local. La comunicación es remota por medio de protocolos, pudiendo elegir el usuario la máquina donde realizar la aplicación, teniendo en cuenta que hay otros usuarios.
 - Lo S.O. realmente o propiamente distribuidos: Aquí no son conscientes de que hay más de un usuario. Permite que el programa se ejecute en distintas C.P.U. (Ejemplo: el Linux de la Escuela).





4.- Soporte Hardware de los S.O.

1) DISTINTOS MODOS DE FUNCIONAMIENTO DE LOS PROCESOS:

Estos modos de funcionamiento del procesador se van a almacenar en 1 ó más bits del programa.

a) Modo supervisor o Kernel (Bit de modo a 0):

- ☑ Tiene acceso completo a todas las direcciones.
- ☑ Puede ejecutar cualquier instrucción máquina.
- ☑ Puede cambiar el valor de cualquier sentencia de la C.P.U.

b) Modo Usuario (Bit de modo a 1):

- ☑ Sólo tiene acceso a un rango de dirección de memoria.
- ☑ No tiene acceso a las instrucciones privilegiadas.
- ☑ Activación o desactivación de las instrucciones.
- ☑ Sólo puede cambiar los valores de algunos registros del procedimiento.
- ☑ Software de diagnóstico.
- ☑ Programas que resuelven estadísticas: para comprobar el funcionamiento de la C.P.U. y hacer mejoras.

Nota: Sólo el S.O. puede usar la instrucción privilegiada de cambio de modo.

2) INTERRUPCIONES Y EXCEPCIONES:

Hardware: Mecanismo por el cual cuando se ha producido un problema o algo raro en el sistema, provoca el paso del control de la C.P.U. a una interrupción que se almacena en una posición de memoria

Cuando se produce una interrupción de hardware, los pasos que sigue el ordenador son los siguientes:

1. Deshabilitar las interrupciones para que sólo se produzca una cada vez (Opcional).
2. Salvar el contador del programa y la palabra de estado (C.P. y P.S.W. respectivamente).
3. Pasar a modo supervisor.
4. Calcular el vector de interrupción.
5. Pasar el control a la R.T.I. (Rutina de tratamiento de información; se carga la 1ª rutina en el vector de interrupción).
6. Una vez ejecutada la R.T.I. restauramos el C.P. (contador de programa).
7. Poner en modo usuario.
8. Pasar el control al proceso que estaba en curso antes de que se produjera la interrupción.

Cuando se produce una interrupción hay que controlar que no se produzca otra ya que al provocarse 2 interrupciones a la vez se puede quedar colgado. Para evitar que se ejecuten 2 interrupciones a la vez y que se quede colgado, existen dos posibles soluciones:

- a. La más burda, que consiste en deshabilitar las interrupciones nada más producirse una para que no se produzcan más (es decir, el paso 1).
- b. Enmascaramiento de interrupciones: es un proceso más común y menos burdo que el anterior. Nos clasifica las interrupciones por tipos (privilegios). En el momento de producirse una interrupción, se deshabilitan las que sean de igual o menor privilegio.

Las interrupciones se utilizan también para controlar unidades que operan aparentemente y para responder a condiciones concretas del procesador.

Los tipos de interrupciones dependen de la arquitectura del procesador como por ejemplo:

- ✓ Ejecución de un programa (overflow,...).
- ✓ Temporizador o reloj.
- ✓ Funciones de E/S.
- ✓ Fallo de hardware.
- ✓ Error en el direccionamiento.
- ✓ Violación de privilegios.
- ✓ Acceso a instrucciones ilegales o privilegiadas.
- ✓ Finalización de la tarea de un controlador.
- ✓ Chequeo de la propia máquina.

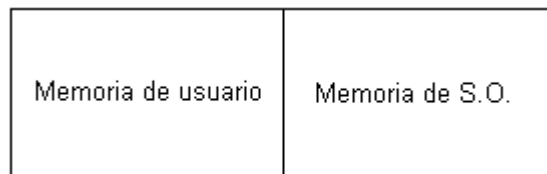
3) PROTECCIÓN:

Este subapartado lo vamos a dividir en 3 partes claramente diferenciadas:

a) Protección de E/S :

Una protección de E/S se asegura haciendo que las instrucciones de E/S sean instrucciones máquina, i.e. que sean instrucciones privilegiadas. El hardware cuida de que dichas instrucciones se ejecuten sólo en modos supervisor ya que si se hace tentativa de hacerlo en modo usuario, este no la ejecuta, la trata como ilegal. Siempre que se produce una interrupción, el hardware cambia de modo usuario a supervisor y antes de devolver el control al programa se vuelve a pasar a modo usuario.

b) Protección de memoria:



En este caso debemos proteger a:

- La memoria que une el S.O. de los procesos de usuario. Con ello obtenemos la protección en la memoria
- Los procesos de usuarios entre ellos.
- Otro hardware subyacente, hardware que use memoria
- El vector de interrupciones de cualquier modificación por parte del usuario (**muy importante proteger esto**).
- Las rutinas de servicio de interrupciones del supervisor residente ante una posible modificación.

Tenemos que asegurarnos que un programa de usuario nunca toma el control del ordenador en modo supervisor

c) Protección de la C.P.U.:

Se basa en garantizar que para cierto intervalo de tiempo se pasará el control al S.O., esto se realiza mediante un reloj que cada cierto intervalo de tiempo produce una interrupción. Con esto tratamos de contrarrestar los bucles infinitos y que un proceso no controle la C.P.U. todo el tiempo.

5.- Visión externa de los S.O.

- 1) Servicios de los S.O.: este apartado lo vamos a dividir en dos subapartados claramente diferenciables:
 - a) Ayuda al usuario:
 - i) Aporta al usuario instrucciones para que este pueda ejecutar sus programas. Ejem.: **ps** Proceso que estaba activo en el sistema, el usuario sabe en que estado están sus procesos.
 - ii) Operaciones de E/S, que no pueden ser ejecutadas por el usuario sino que es el S.O. el que proporciona las instrucciones en orden
 - iii) Manipulación de archivos: abstracción de archivos. El S.O. nos da el concepto de archivo.
 - iv) Respecto a la comunicación entre procesos: cuando tenemos distintas máquinas. Tiene 2 implementaciones básicas:
 - (1) Memoria compartida. Puede usarse cuando 2 procesos están en la misma memoria, que trabajan sobre la misma variable.
 - (2) Paso de mensajes. Un proceso manda una señal a otro proceso (por ejemplo para sincronizarlos).
 - v) Detección de errores: el S.O. debe ser capaz de detectar y corregir cualquier tipo de error, tanto:
 - (1) Hardware: C.P.U., memoria, dispositivos, etc,
 - (2) Programas de usuario
 - vi) Gestión del sistema: no son generalmente servicios para usuarios normales sino que son servicios que están para los administradores.
 - b) Correcto funcionamiento del S.O.:
 - i) Realiza una asignación de recursos.
 - ii) Contabilidad: permite medir cuanto tiempo de C.P.U. ha consumido un determinado usuario (sobre todo se usa en redes). La contabilidad en este caso tiene 2 fines:
 - (1) Los contables propiamente dichos
 - (2) Los estadísticos.
 - iii) Relativos a protección: deben dar mecanismos para proteger a los usuarios entre ellos mismos.
- 2) Usuario del lenguaje de órdenes (SHELL)

Son los que obtienen servicios del S.O., tecleando órdenes desde su terminal o a través de un trabajo por lotes.

El interprete (traductor) de órdenes captura dichas órdenes introducidas por el usuario, las traduce y ejecuta las llamadas al sistema necesarias para recibir el servicio requerido por el usuario.

Cuando el usuario está trabajando con el sistema, el intérprete de órdenes (SHELL) se desconecta mientras se ejecuta la orden.

Cuando se ejecuta una orden en background (segundo plano), antes de que termine la ejecución aparece el prompt del sistema en pantalla. Ejemplo: **ls>fich &**

- ☞ Background: Interactivamente, devuelve el prompt para seguir mientras hace algo.
- ☞ Foreground: El programa se espera hasta que finaliza una orden.

3) Usuarios de las llamadas al S.O.:

Son los que realizan las llamadas al S.O. en tiempo de ejecución. Estas llamadas van a realizar una interface entre los programas de usuario en ejecución y el S.O.

Estas llamadas pueden ser mediante interrupciones o también mediante funciones que realizan los pasos necesarios para que hagan las llamadas al sistema.

El mecanismo para iniciar una llamada al S.O. transcurre por los siguientes pasos:

- i) Ejecución de una instrucción en Ensamblador. Ejemplo: **int 12** (abrir un fichero).
 - (1) Llamada al sistema b Interrupción SINCRONÍA.
 - (2) Interrupción hardware b ASINCRONA. (se produce al hacer una llamada al sistema).

- ii) Llamada a una función

Si mientras yo hago una llamada al S.O. con una de las formas anteriores, si se produce un error, ello provoca una interrupción.

La información que se debe transmitir al S.O. es:

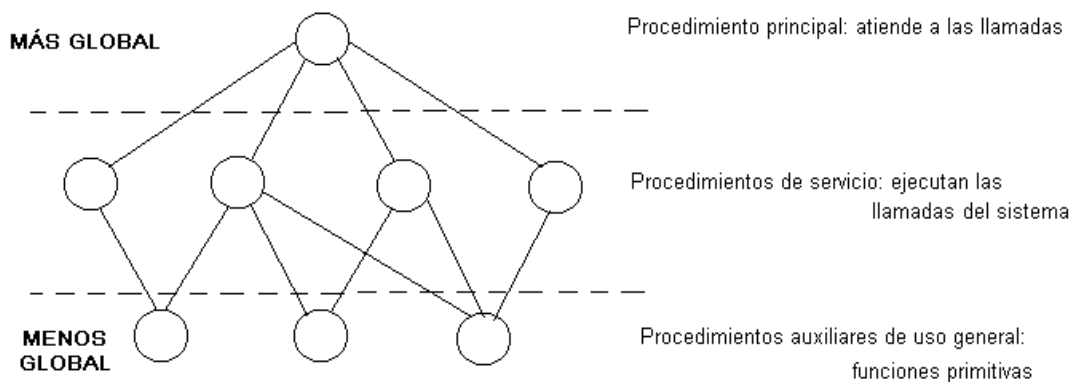
- (1) Registro de la C.P.U.
- (2) Zona de memoria.
- (3) Pila (esto no lo puede hacer el propio usuario).

A su vez el S.O. debe devolver el resultado y eso puede hacerse en alguno de los sitios anteriores. Para esto Unix tiene una instrucción que es **trap**.

6.- Estructura interna de un Sistema Operativo.

1. Sistemas monolíticos.

Se caracterizan porque no tienen estructura o si la tienen está poco definida. Se trata de un conjunto de procedimientos, los cuales están visibles entre ellos. No hay modularidad ni abstracción de la información. Sólo hay un programa objeto. Podemos ver en una pequeña división que los sistemas monolíticos se dividen en tres partes:



Cualquier Procedimiento puede llamar a cualquier otro.

2. Sistemas por capas o por niveles.

Se trata de una degeneración de los sistemas monolíticos. Se dividen por capas y cada una de ellas tiene una función bien definida. Cada capa a su vez se puede dividir en un conjunto de módulos.

Las ventajas del sistema por capas respecto del anterior son las siguientes:

- ↳ Modularidad.
- ↳ Fácil mantenimiento.
- ↳ Fácil modificación de alguna de las capas (los cambios no afectan a las otras capas).
- ↳ Fácil detección de errores.

Las desventajas de los sistemas por capas o la dificultad aparece a la hora de crear dichas capas, al igual que la jerarquía de dichas capas, i.e. saber cual es superior o prioritaria y cual es inferior.

La gestión de memoria no va con el hardware desnudo, sino que se has de planificar en procesos. Modificar cualquier procedimiento de una capa es muy fácil sin tener los interfaces de las capas anteriores.

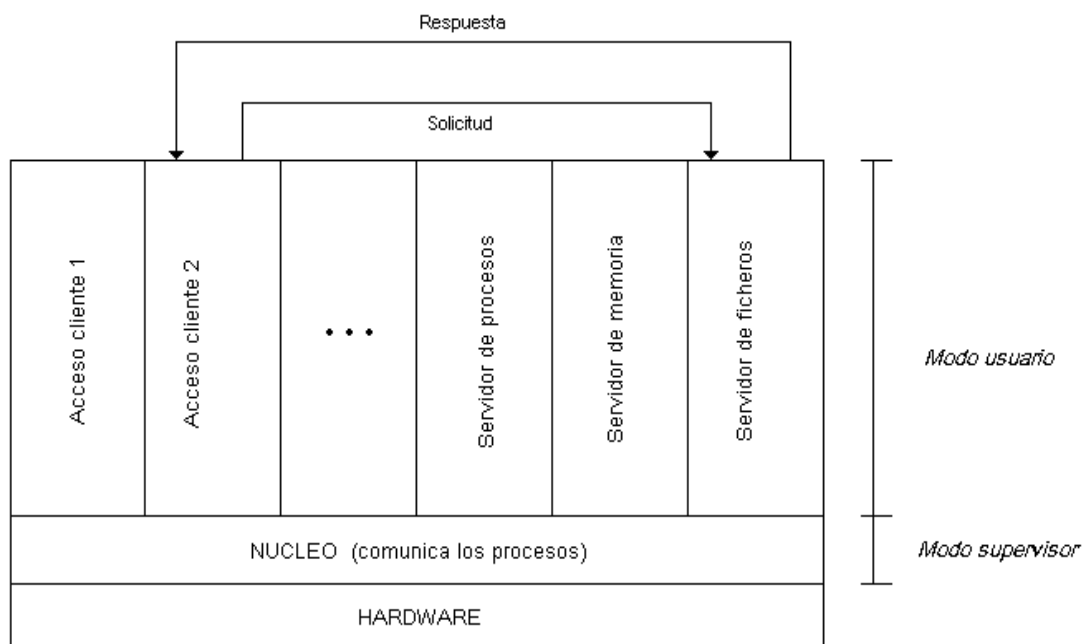
Tienen 6 capas:

1. Situada sobre el hardware: se asigna al procesador y gestores de multiprogramación
2. Gestión y administración de memoria y disco: contiene las direcciones de los procesos en memoria y disco.
3. Común: entre operador y procesos
4. Manejadores o controladores de dispositivos E/S.
5. Programas de usuario
6. Proceso que se está ejecutando en ese momento.

Ejemplo: el sistema THE.

3. Sistemas cliente – servidor.

Estos sistemas se basan en implementar partes del S.O. como procesos independientes y conservar un núcleo mínimo.



Ventaja: al ejecutarse los servidores en modo usuario, si algo falla no tiene porque pararse la máquina ya que se ejecuta en modo usuario.

Surge en este momento un problema que consiste en que si implemento el servidor de disco en modo usuario, no puede acceder a las operaciones de E/S y cosas parecidas. Este problema tiene 2 posibles soluciones:

- 1) El núcleo se encarga de realizar las acciones en modo supervisor. Los servidores se encargan de la gestión.
- 2) Permitir que los servidores críticos se ejecuten en modo supervisor. Pero las comunicaciones con los clientes se realizan a través del núcleo. Así quedaría:

Cliente solicita un servicio b Núcleo b Servidor lo sirve

Existen sistemas distribuidos que utilizan una arquitectura de cliente – servidor, como por ejemplo AMOEBA, MACA, etc.

NFS: Network File System

Tema 2: Procesos

1.- Concepto de proceso

Definición: Es el programa en ejecución y más cosas, i.e. un proceso es una entidad activa, que está en ejecución mientras que un programa es una entidad pasiva que no está en ejecución.

El S.O. se encarga de realizar una planificación de la C.P.U. entre los distintos tipos de procesos de manera que parezca que cada proceso dispone de una C.P.U. virtual.

Por tanto un proceso:

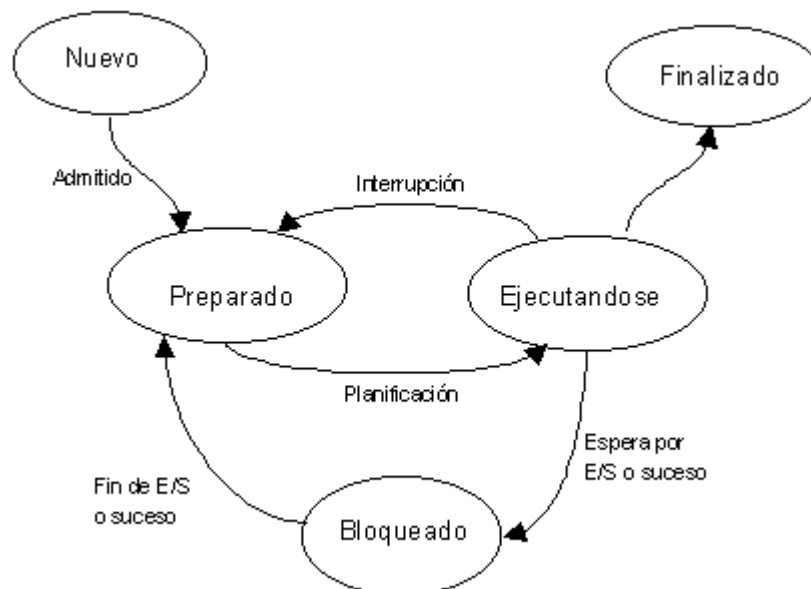
- ☞ Es un programa en ejecución.
- ☞ Controla los datos y la pila.
- ☞ El contexto de un proceso incluye los valores activos del C.P., registros del procesador y variables del programa.

Un proceso puede encontrarse en los siguientes estados:

- x *Ejecutándose o activo:* Cuando utiliza la C.P.U. en un instante dado (sólo puede ser un proceso a la vez).
- x *Ejecutable, preparado o listo:* Se encuentra detenido a la espera de que acabe de ejecutarse otro proceso para usar la C.P.U.
- x *Bloqueado:* No se puede ejecutar debido a que se ha producido algún evento extraño.

Los procesos son asíncronos e independientes unos de otros. En la C.P.U. sólo se ejecuta un proceso cada vez.

Diagrama de transposición de estados:



1. *El proceso se bloquea en espera de datos:* Esto ocurre cuando el proceso descubre que no puede continuar. En algunos sistemas hay que hacer una llamada al ídem para pasar al estado de bloqueo.
2. *El planificador elige otro proceso:* Esto se produce cuando el planificador decide que el proceso en ejecución ya ha sido ejecutado el tiempo suficiente y que es hora de que otro proceso tenga tiempo de C.P.U.

3. *El planificador elige este proceso:* Ocurre cuando los demás procesos han tenido su parte y este proceso vuelve a ejecutarse.
4. *Los datos están disponibles:* Se produce cuando ocurre el evento externo por el cual espera el proceso (ejemplo: entrada de datos nuevos). Si no existe otro proceso en ejecución en ese momento se produce la transición a **ejecutándose** de forma inmediata.

Cuando a uno de los procesos se le retira la C.P.U., cada vez que pasa de bloqueado a ejecutable o al revés, el S.O. debe guardar toda la información necesaria del proceso que luego vuelve al mismo lugar donde estaba.

Toda la información que debe guardar el S.O. lo hace en una tabla de procesos. En esta tabla hay una entrada por cada proceso que exista en el sistema y contiene la información relativa al estado del proceso. A cada una de estas entradas se le llama descriptor de proceso o P.C.B. En el se guarda la información necesaria para que cuando llamemos nuevamente a un proceso que previamente le hemos retirado la C.P.U. pueda continuar. A esto se le llama cambio de contexto.

Este P.C.B. (Bloque de Control de Procesos) guarda:

1. P.I.D.: identificador de procesos que contiene
 - a. Nombre del proceso.
 - b. Nombre del usuario.
 - c. U.I.D.(identidad del usuario).
2. Estado actual del proceso: ejecutable, ejecutándose o bloqueado.
3. Guarda la línea o el puntero al proceso padre o a los procesos hijos, lo que corresponda, en los sistemas con jerarquía de procesos.
4. Los sistemas que asignen prioridad a los procesos guardarán dicha prioridad.
5. Información relativa a la memoria propia del proceso, así como un puntero a dicha memoria.
6. C.P., registros del proceso (valores).
7. Procesador en el que se ejecuta (para sistema multiprocesadores).
8. Información acerca de los recursos que usa el proceso:
 - a. Ficheros que tiene abiertos.
 - b. Peticiones de E/S pendientes o dispositivos de E/S que tiene asignados.
9. Información estadística:
 - a. Tiempo de C.P.U. consumido por el proceso.
 - b. Volumen de E/S.
 - c. Tiempo bloqueado.

Nota: una interrupción no provoca un cambio de contexto. Cuando acaba la interrupción se sigue con el proceso que estaba antes de dicha interrupción.

A cada clase de dispositivos de E/S se les asocia una localidad cercana a la parte inferior de la memoria, llamado vector de interrupciones que contiene la dirección del procedimiento de servicio a las interrupciones si se produce un cambio de contexto (ejemplo: interrupción de disco).

Este cambio de contexto es una operación que normalmente sufren los procesos y consiste en retirar la C.P.U. al proceso actual (guardando todos los registros en la tabla de procesos). Como variables globales se pasan el n° de proceso activo y el apuntador de datos. Seguidamente la información depositada por la interrupción se elimina de la pila y el apuntador de ésta apunta a una pila temporal que utiliza el control del proceso. Por último asignamos la C.P.U. a otro proceso (ejecutable)

Hay dos tipos de procesos:

- i. *Limitados por E/S*: son procesos que usan la C.P.U. en intervalos cortos y pocas veces y usan mucho E/S.
- ii. *Limitados por la C.P.U.*: procesos que usan mucho la C.P.U. y poco E/S.

Para obtener el mayor rendimiento posible del sistema, debemos mantener un equilibrio entre ambos tipos de procesos.

2.- Planificador de procesos.

El planificador es el que se encarga de decidir por ejemplo que proceso se va a ejecutar con anterioridad si hay más de uno ejecutable, es decir, es la parte del S.O. que planifica el orden de ejecución de los programas y utiliza un algoritmo llamado algoritmo de planificación.

Dentro de este apartado vamos a ver varios subapartados:

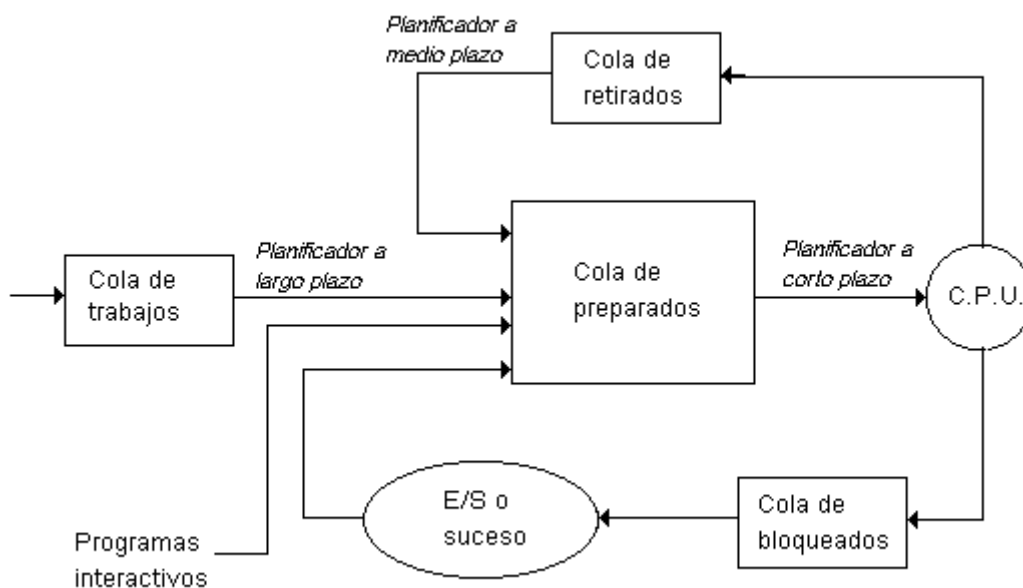
a) Tipos de planificadores.

La actividad básica del planificador consiste en controlar que trabajos entran en un sistema o que procesos interactivos se van a ejecutar en un momento dado. El objetivo principal es *optimizar* el rendimiento del sistema.

Existen 3 tipos de planificadores:

- i. *Planificadores a largo plazo*: son los que se encargan de trabajar con la cola de trabajos por lote (batch). Se encarga de admitir en el sistema los trabajos por lote. Se llama cuando un proceso finaliza o si no finaliza ninguno en un cierto tiempo.
- ii. *Planificadores a medio plazo*: se encargan de procesar los trabajos que están bloqueados y lo que va a insertar y quitarlo de memoria para ponerlos en la cola de ejecutables.
- iii. *Planificadores a corto plazo*: de los procesos que están en la cola de ejecutables, este planificador elige que proceso va a asignarlo a la cola de la C.P.U.

Diagrama:



El planificador a largo y medio plazo es el que se encarga de regular la carga del sistema mientras que el planificador a corto plazo es el que se encarga de planificar lo mismo.

En los sistemas operativos pequeños sólo existe el planificador a corto plazo.

b) Dispatcher o despachadores.

Es el que se va a encargar realmente de mandar el control de la C.P.U. a un proceso de la cola de ejecutables. Se encarga del planificador a corto plazo y asigna procesos a la C.P.U.

El funcionamiento de esto tiene varios pasos:

- a. Decide si se debe cambiar el proceso que está activo.
 - i. Si se debe cambiar b Paso b.
 - ii. Si no se debe cambiar b Devuelve el control al proceso activo
- b. Salva el entorno volátil del proceso actual.
- c. Toma el primer proceso de la cola de preparados o ejecutables.
- d. Carga el entorno volátil del proceso elegido (el contenido del P.C.B.).
- e. Cede el control al proceso y lo pone en modo usuario.

La actuación del despachador se invoca por alguna de las siguientes razones:

- ↳ Un proceso finaliza.
- ↳ El S.O. decide que el proceso activo no debe seguir ejecutándose.
Ejemplo: E/S o espera a la finalización de los procesos hijos.
- ↳ El proceso agota su quantum de tiempo.
- ↳ Un proceso cambia de estado de bloqueado a ejecutable.

c) Políticas de planificación de la C.P.U.

Se implementa con vistas a conseguir el mayor rendimiento de la C.P.U. Consiste en un algoritmo para planificar o controlar la asignación de la C.P.U., ya que puede haber varios usuarios peleándose por la C.P.U. y el planificador debe decidir que proceso es más importante de los varios que están en la cola de ejecutables. Los objetivos son dos:

1. Obtener un buen rendimiento, i.e. que se haga el mejor trabajo útil posible en el sistema.
2. Obtener un buen servicio.

Con objeto de establecer si estamos obteniendo o no un buen servicio de manera objetiva establecemos unas medidas. Partamos de un proceso P_1 que necesita un tiempo de servicio t .

- ☞ **T (tiempo de respuesta):** tiempo transcurrido desde que se remite una solicitud hasta que se produce la primera respuesta.
- ☞ **M (tiempo de espera):** tiempo que un proceso ha estado esperando en la cola de preparados ($T-t$).
- ☞ **P (penalización):** tiempo que ha desperdiciado el proceso en cada ráfaga (T/t). Si **P** es bajo mejor porque el proceso se va a ejecutar en menos t de C.P.U.
- ☞ **R (índice de respuesta):** es la fracción de tiempo que el proceso P_1 está recibiendo servicio (t/T). Si **R** se aproxima a 1 será bueno mientras que si se aproxima a 0 será malo.
- ☞ **Tiempo de núcleo:** tiempo perdido por el núcleo del S.O. tomando decisiones de planificación y cambio de contexto. Típicamente tiene un valor de entre el 10% y el 30% del tiempo consumido.
- ☞ **Tiempo de inactividad:** es el tiempo en que la cola de procesos ejecutables está vacía y no se realiza ningún trabajo productivo.

Podemos clasificar los procesos en 2 tipos diferenciados:

1. *Procesos cortos*: aquellos procesos con fuerte demanda de E/S y baja demanda de C.P.U.
2. *Procesos largos*: aquellos procesos con fuerte demanda de C.P.U. y baja demanda de E/S.

No existe ninguna política de planificación que favorezca a todos los procesos en general.

Igualmente podemos clasificar los algoritmos de planificación en 2 categorías:

1. *No apropiativos o sin desplazamiento sin derecho preferente*: son aquellos que mantienen el proceso al cual se le ha asignado la C.P.U. en ésta hasta que se bloquee o finalice.
2. *Apropiativos o con desplazamiento o con derecho preferente*: son aquellos que pueden retirarle la C.P.U. a un proceso aunque éste no se haya bloqueado o finalizado.

A priori no podemos conocer cuanto va a durar la ejecución de un proceso por lo que únicamente podemos realizar una estimación. Se podría seguir una fórmula de promedio exponencial que se va a basar en las longitudes ya medidas de las ráfagas de C.P.U. anteriores:

- T_N : duración real de la n – ésima ráfaga.
- $*_n$: valor estimado de la n – exima ráfaga.
- $,$: valor de ponderación. $0 \leq , \leq 1$

Así pues nos quedaría:

$$\Psi_{n+1} = \alpha \cdot T_n + (1 - \alpha) \cdot \Psi_n$$

Normalmente, $, = 0.5$. En el caso de que $, = 0$ obtendríamos que la historia reciente no influye y en el caso de que $, = 1$ veríamos que solo cuenta la ráfaga actual.

Otra forma de planificar los procesos en espera es mediante un algoritmos de prioridades, el cual le asigna un número entero que indica la prioridad del proceso. Hay dos formas de asignar las prioridades:

- ✓ Internamente b S.O. ▶ b Estática
- ✓ Externamente b Usuario ▶ b Dinámica.

Los algoritmos se pueden implementar de dos formas.

- ☐ *Apropiativa*: Permite que los procesos ejecutables (desde el punto de vista lógico) sean suspendidos de forma temporal.
- ☐ *No apropiativa*: Se ejecuta el proceso seleccionado hasta que termine. Estos sistemas son fáciles de implementar aunque no son adecuados para los sistemas de propósito general con varios usuarios.

Problema: Puede pasar que con un algoritmo apropiativo, un proceso se bloquee indefinidamente.

Solución: El envejecimiento de los procesos. Se basa en aumentar la prioridad de un proceso ha medida que transcurra el tiempo para que de esta forma no se bloquee indefinidamente y no nos de problemas de inanición.

3.- Operaciones sobre procesos.

A. Creación: consiste en coger memoria principal y definir la estructura de datos del proceso así como asignar el espacio de direcciones propio.

- a. Primero se le asigna un nombre al proceso. Puede ser simplemente con un identificador de procesos (P.I.D.).
- b. Creación del P.C.B. o descriptación de procesos. Inserta el proceso en la tabla de procesos.
- c. Asigna alguno de los valores a los contenidos del P.C.B.:
 - i. Establece la prioridad inicial.
 - ii. Establece el estado inicial.
- d. Asigna recursos iniciales como memoria, archivos abierto, etc.

El S.O. crea un proceso debido a 4 situaciones posibles:

1. Recepción y admisión de un nuevo trabajo (sistemas batch).
2. La conexión del usuario provoca la creación de un proceso que ejecute el SHELL (sistema interactivos).
3. El S.O. crea un proceso en respuesta a una petición del usuario.
4. Un proceso puede solicitar la creación de otro proceso.

Existen S.O. que permiten jerarquías de procesos, i.e. se permite que un proceso genere otro proceso, creando una estructura en árbol de tal forma que el proceso que genera en él procesos se llama padre y los generados hijos.

Cuando se produce una jerarquía de procesos, caben dos posibilidades:

- ☞ Los hijos obtienen recursos directamente del S.O.
- ☞ Los hijos comparten los recursos del padre.

Una vez que el proceso padre ha creado a los hijos y este comienza a ejecutarse, esta forma de ejecución puede ser:

- ↳ El proceso padre se bloquee hasta que finalicen sus hijos (foreground).
- ↳ Tanto el proceso padre como el hijo se ejecutan concurrentemente (UNIX). (background).

ESPACIO DE DIRECCIONES:

- x *Alternativa 1:* Cuando se crea un proceso, existen sistemas donde el hijo es una copia exacta de la imagen del padre (UNIX: va a hacer una llamada al sistema (**fork**), por tanto todos los recursos del padre van a pasar al hijo).
- x *Alternativa 2:* Se crea el hijo con un programa que carga el propio fichero hijo (VMS, que es un S.O.).

B. Destrucción: un proceso puede terminar por:

- a. Ha finalizado. Es algo que el propio proceso se encarga de notificar al S.O.
- b. El proceso provoca la finalización de otro proceso. Esto es privilegio de los procesos padres sobre sus hijos.

Eliminarlos del S.O. significa:

- ♣ Borrarlos de cualquier lista o tabla del S.O.
- ♣ Borrar el P.C.B. de la tabla de procesos.
- ♣ Desasignar los recursos que estaban activos.

En un sistema de jerarquías de procesos como el anterior puede ocurrir:

1. Llamada al sistema (**exit**) para eliminar los recursos del sistema.
2. Dos casos posibles:

- a. Si el proceso padre finaliza, tanto el proceso padre como el proceso hijo concluyen. La terminación del padre b terminación del hijo.
- b. Si el padre termina, los hijos continúan ejecutándose de forma implícita. El hijo hereda los recursos del padre caso de estar compartiéndolos.

Otra forma de terminación del sistema es con **abort**. Un proceso puede finalizar por concluir una ejecución o porque un proceso de jerarquía mayor realiza una llamada al sistema, pero acaba con él.

4.- Threads (hebras o hilos).

Sería bueno tener unos flujos de control de procesos. Supongamos que el S.O. con servidor de ficheros sería un proceso que cuando recibe la solicitud de otro proceso, el servidor de ficheros debe darle paso mientras los otros procesos deben esperar (bloqueado).

Esto se mejoraría con varios flujos de control que comparten el mismo espacio de direccionamiento de forma que si se ejecuta un proceso y se bloquea uno, los demás pueden seguir ejecutándose por otro lado.

Hebras o procesos ligeros: una hebra es un proceso que tiene sus propios

- ☐ Contador de programa.
- ☐ Pila.
- ☐ Conjunto de registros.

Una hebra se puede ver como un proceso de forma que el planificador trabajaría con hebras. Esto mejora el cambio de contexto que sería cambio entre hebras, con lo que se mejoraría el coste.

Todas la hebras van a tener acceso a las divisiones del espacio de direcciones, no hay protección de memoria. Cada hebra comparte con una hebra par:

- ✖ La sección de código.
- ✖ La sección de datos.
- ✖ Recursos del S.O.

Esto puede dar lugar a conflictos a la hora de ejecutarse procesos concurrentes.

Tarea: Entorno clave donde se ejecutan varias hebras que comparten el espacio de direcciones.

El proceso pesado o tradicional es un espacio de direcciones virtual con una única hebra. Las hebras van a poder estar en los mismos estados que le preceden:

- ☞ Ejecutándose.
- ☞ Ejecutable.
- ☞ Bloqueado.

Esto es así porque en un sistema de hebras, se tratan las hebras como unidad básica. Si cambiamos las tareas, empleamos el mismo coste que los procesos paralelos.

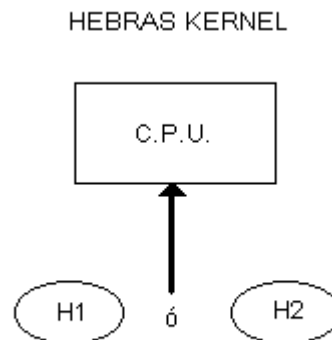
Utilizar el mecanismo de hebras aporta ventajas como:

- ☐ Mayor rendimiento.
- ☐ Aumento de concurrencia (permite varios procesos paralelos concurrentemente).
- ☐ Mejor servicio (para el usuario).

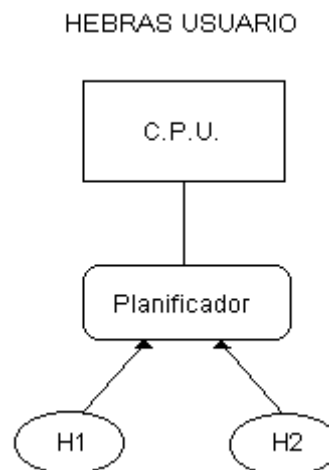
Los sistemas de hebras son muy utilizados en sistemas distribuidos multiprocesador, ya que podemos ejecutar una hebra en cada procesador.

Podemos observar varios niveles de hebras:

1. *Hebras implementadas en el Kernel:* el kernel tiene implementadas las rutinas necesarias para el tratamiento de las hebras. Entra en la C.P.U. una hebra u otra.



2. *Hebras a nivel de usuario:* las rutinas para trabajar con las hebras están implementadas en librerías, no en el kernel. La C.P.U. se reparte entre las diferentes hebras.



Tema 3: Comunicación y sincronización

1.- Concurrencias.

Procesos concurrentes: son aquellos procesos cuya ejecución se solapa en el tiempo.

Los procesos mantienen 2 tipos de relaciones:

1. Relación de cooperación: los procesos concurrentes forman parte de la misma aplicación.
2. Relación de competición: cuando los distintos procesos compiten por un recurso.

El software que se ocupa de los procesos concurrentes es la programación concurrente, que va a abarcar al conjunto de técnicas y rotaciones necesarias para expresar el paralelismo potencial de un sistema concurrente y resolver los problemas de sincronización y comunicación.

Existe paralelismo cuando hay más de una C.P.U. y pseudo – paralelismo cuando solamente existe una C.P.U.

Razones por las que se necesita concurrencia en un sistema:

1. *Compartir recursos físicos*: pues estos son limitados en un sistema. Para dar servicio a varios usuarios los procesos han de compartir recursos si dichos procesos se ejecutan de forma simultánea.
2. *Compartir recursos lógicos*: nos puede interesar que dos procesos compartan un fichero; para ello hay que utilizar los mecanismos necesarios que nos eviten problemas.
3. *Mayor velocidad de ejecución*: si tenemos un proceso que realiza una adecuación de una lista de números, se podría conseguir una ejecución más rápida si se divide el problema en 2 partes. En el caso de existir paralelismo real la ejecución del proceso mediante la realización de divisiones de este será más rápida que de la otra forma.
4. *Por razones de modularidad*: vamos a tener diversos procesos independientes que realizan sus propias tareas.
5. *Comodidad*: desde el punto de vista del usuario, por poder realizar varias tareas simultáneas.

Programas concurrentes: Existen varias secuencias que se ejecutan en paralelo, por tanto existen varios flujos de control. Existen lenguajes de programación concurrente que permiten dicha programación.

Dentro de un sistema operativo se distinguen los siguientes procesos:

- ✧ *Procesos independientes*: procesos que se ejecutan independientemente y no necesitan información compartida, esto es un proceso que no va a depender de otro
- ✧ *Procesos cooperativos*: procesos que comparten información con otros. Su estado puede ser afectado por la ejecución de otro. Ejemplo: proceso productor / consumidor.

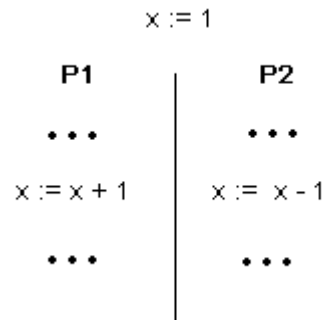
Ejemplo de procesos cooperativos: productor – consumidor

Se dispone de un buffer en el cual introduce datos un proceso productor y un proceso consumidor retira los datos del mismo. Estos procesos son cooperativos porque se ejecutan de forma concurrente. Por ejemplo, si el proceso productor ha llenado el buffer, este proceso ha de esperar hasta que el proceso consumidor retire datos de este. El consumidor no puede retirar datos del buffer si previamente el productor no ha introducido nada. Es necesaria la introducción de mecanismos de concurrencia y sincronización para controlar la ejecución concurrente.

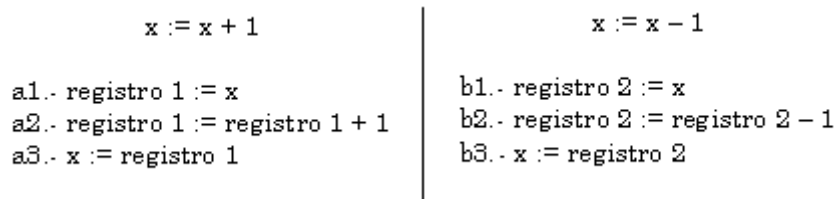
Otra característica de los procesos concurrentes será por tanto la sincronización entre procesos que se ejecutan de dicha forma.

Al modo en el que se ejecutan los procesos, esto es, la secuencia de instrucciones que se realizan durante la ejecución de un proceso se denomina **interfoliación**. Esta será una característica de los sistemas concurrentes.

Ejemplo: supongamos que se ejecutan concurrentemente dos procesos P1 y P2, compartiendo ambos una variable común **x**.



X está inicialmente a uno. ¿Qué valor tendrá la variable **x** después de que se hayan ejecutado ambos procesos? ¿0, 1 ó 2?. La clave está en que las instrucciones $x := x + 1$ y $x := x - 1$ no son indivisibles. Cada una se implementa en lenguaje máquina como:



Registro 1 y registro 2 son registros del procesador (incluso pueden ser el mismo físicamente: un acumulador).

Veamos distintas secuencias de ejecución:

☞ a1 b1 a2 b2 a3 b3	Resultado: x=0.
☞ a1 a2 a3 b1 b2 b3	Resultado: x=1.
☞ a1 b1 b2 b3 a2 a3	Resultado: x=2.

Conclusiones:

1. Debemos asegurarnos que la ejecución de procesos concurrentes sea correcta independientemente del orden en que se ejecuten sus instrucciones (interfoliación).
2. No podemos hacer suposiciones de la velocidad a la que evoluciona cada proceso. Si no se utiliza un mecanismo especial no se pueden realizar suposiciones de ningún tipo.

3. Necesitamos asegurar que sólo un proceso en cada ocasión manipule la variable compartida. Para ello necesitamos algún tipo de **sincronización** de procesos, establecer un orden relativo dentro de la interfoliación

2.- Sincronización.

La sincronización es la paralización de una actividad mientras no se cumpla una determinada condición.

Ejemplo: supongamos que queremos calcular la siguiente expresión

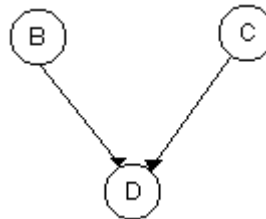
$$A = x + 3 \cdot z + (y + v)$$

y tras ello, escribir el valor de A. Podríamos construir dos procesos concurrentes que realizaran lo siguiente:

P1	P2
B.- temp1 := x+3·z	C.- temp2 := y + v
	D.- A := temp1 + temp2; write (A);

Se observa que B y C se pueden ejecutar concurrentemente pero D se ha de ejecutar después. Hay que sincronizar P1 y P2, de tal forma que P2 ha de esperar hasta que P1 haya finalizado su ejecución.

Para la secuenciación de las operaciones a realizar se utilizan **grafos de precedencia**, que son grafos dirigidos acíclicos, en los cuales, los nodos representan las instrucciones.



Dependiendo del problema al que nos enfrentemos tendremos una solución u otra. Así pues podemos distinguir 2 casos:

- ♣ *Memoria común:* usamos espera ocupada. Consiste en utilizar una variable booleana para que un proceso espere hasta que el otro acabe. Así mismo podemos utilizar semáforos cuyo procedimiento es el mismo pero están predefinidos por el sistema.
- ♣ *Mensajes:* cuando no se pueden compartir memoria.

Problema de la exclusión mutua.

Un problema particular de la sincronización se plantea cuando en un momento dado, dos o más procesos intentan acceder a un recurso común (datos compartidos). La solución consiste en asegurarse que la secuencia de sentencias que acceden a datos compartidos se traten como una operación indivisible.

Sección crítica: Secuencia de sentencias de un proceso que accede a datos compartidos.

REQUERIMIENTOS QUE SE HAN DE SATISFACER DURANTE LA EJECUCIÓN DE LA SECCIÓN CRÍTICA:

1. Dos procesos no deben encontrarse al mismo tiempo dentro de sus secciones críticas b se debe respetar la exclusión mutua. Las secciones críticas deben ejecutarse como procesos individuales.
2. No se deben hacer hipótesis sobre la velocidad de los procesos o el número de C.P.U.'s.
3. Ningún proceso que esté fuera de su sección crítica puede bloquear a otros procesos.
4. Propiedad de vivacidad: debe haber una ausencia de inacción, ningún proceso debe de quedarse retrasado indefinidamente.
5. Ningún proceso debe esperar eternamente para entrar en su sección crítica.
6. Propiedad de equidad: todos los procesos deben avanzar con iguales condiciones.
7. Ausencia de interbloqueo: un interbloqueo es un estado en el cual todos los procesos se esperan unos a otros y el sistema se detiene. Ejemplo: el problema de los 5 filósofos.

Soluciones para implementar la exclusión mutua:

1) Soluciones software:

Protocolo de entrada: antes de ejecutar las instrucciones de la sección crítica se comprueba si se puede entrar a dicha sección, atendiendo al problema de la exclusión mutua.

Protocolo de salida: deja al proceso en un estado de tal forma que cualquier otro proceso puede acceder a su sección crítica, esto es, a la sección crítica del proceso que quedó en este estado.

Entre procesos que comparten datos o acceden a datos compartidos, existen condiciones de competencia. El resultado final de la ejecución dependerá de que proceso se ejecute y de cómo realice su ejecución.

La solución al problema de exclusión mutua es hacer que las operaciones se realicen de un solo paso, de forma indivisible.

Al conjunto de instrucciones que se ejecutan de forma independiente se denomina sección crítica. Sólo un proceso puede entrar en un instante determinado en la sección crítica.

ALGORITMO 1: Alternancia estricta de una variable compartida.

P1 While turno = 2 do no - op; Sección crítica turno := 2; Sección restante	P2 While turno = 1 do no - op; Sección crítica turno := 1; Sección restante
--	--

Desventajas:

✕ Espera ocupada (tiempo de C.P.U. desperdiciado).

- ✕ No cumple la condición 3ª, pues procesos que se encuentran fuera de sus secciones críticas bloquean a otros.
- ✕ Alternancia estricta.

ALGORITMO 2: Variables puerta de paso o de cerradura.

P1	P2
While c2 = 0 do no - op;	While c1 = 0 do no - op;
c1 := 0;	c2 := 0;
Sección crítica	Sección crítica
c1 := 1;	c2 := 1;
turno := 2;	turno := 1;
Sección restante	Sección restante

Si de entrada hago $c1 = c2 = 1$ consigo equidad.
 $c1 = 0$ bP1 está dentro de su sección crítica.
 $c2 = 0$ bP2 está dentro de su sección crítica.
 $c1$ y $c2$ a la vez a cero b No entran en su sección crítica.
 No resuelve el problema de la exclusión mutua.

MODIFICACIÓN DEL ALGORITMO 2:

P1	P2
c1 := 0;	c2 := 0;
While c2 = 0 do no - op;	While c1 = 0 do no - op;
Sección crítica	Sección crítica
c1 := 1;	c2 := 1;
Sección restante	Sección restante

Inicialmente $c1 = c2 = 1$.
 Ningún proceso entra en la sección crítica, pero si soluciona el problema de la exclusión mutua.
 Pueden bloquearse ambos procesos b INTERBLOQUEO.

ALGORITMO 3: Modificación de los dos anteriores.

Cambia las estrategias anteriores, utiliza una variable turno, que controla cuando 2 procesos intentan acceder a la vez a la sección crítica.

$c1$ y $c2$ indican que la sección crítica está libre para P1 ó P2, esto es, si P1 está en la sección crítica $c1$ está a cero y en caso contrario sale de la sección crítica.

Se cumple la condición de la exclusión mutua.
 Se cumple la condición 3ª, esto es, ningún proceso puede bloquear a otro si no se encuentra en la sección crítica.
 Cumple todas las condiciones.

P1	P2
c1 := 0;	c2 := 0;
turno := 2;	turno := 1;
While (c2 = 0 and turno = 2) do no - op;	While (c1 = 0 and turno = 1) do no - op;
Sección crítica	Sección crítica
c1 := 1;	c2 := 1;
Sección restante	Sección restante

2) Soluciones hardware:

a) Activación – desactivación de interrupciones

P1	P2
Desactivar interrupciones	Desactivar interrupciones
Sección crítica	Sección crítica
Activar interrupciones	Activar interrupciones
Sección restante	Sección restante

Si se desactivan las interrupciones no se presenta el problema de la exclusión mutua porque al hacerlo impedimos que ningún otro proceso pueda parar al que ya está.

Desventajas:

- ☒ No se debe permitir a los procesos de usuario utilizar este mecanismo de exclusión mutua, que será utilizado únicamente por procesos del S.O.
- ☒ Excluye a todos los procesos del sistema, no sólo a los procesos que poseen las condiciones de competencia.
- ☒ Si existe un bloqueo en la sección crítica se para el sistema debido a que se han desactivado las interrupciones.
- ☒ Los mecanismos para implementar sincronización y comunicación son de bajo nivel.
- ☒ Si tenemos más de un C.P.U. no nos sirve la activación – desactivación de interrupciones porque sólo se realiza esto para un procesador y los demás no se ven afectados. Por tanto falla en sistemas multiprocesadores.

b) Instrucción test & set

```
Function TestAndSet (var objetivo: boolean) : boolean;

    Begin
        TestAndSet := objetivo;
        objetivo := true;
    End;
```

P1	P2
While test – and – set (cerradura) do no – op;	While test – and – set (cerradura) do no – op;
Sección crítica	Sección crítica
cerradura := falso;	cerradura := falso;
Sección restante	Sección restante

Cerradura inicialmente tiene asignado el valor falso.

En este caso se dispone de una interrupción llamada *TEST & SET* que se ejecuta de forma indivisible, realizando la devolución de un valor booleano. Dado un parámetro, se actualiza devolviendo la variable objetivo y asignándole a ésta el valor verdadero.

Mientras *TEST & SET* sea '1' no se entra en la sección crítica.

Ventajas: soluciona el problema de la exclusión mutua en sistemas multiprocesadores.

Inconvenientes:

- ♣ si se bloquea en la sección crítica, se bloquean todos los procesos: interbloqueo.
- ♣ Utiliza la espera ocupada b perdemos eficiencia.

Existe otra instrucción llamada *TEST & SET BLOCK* que a diferencia de la anterior realiza intercambio de variables.

3.- Semáforos.

Un semáforo es un tipo de datos que sólo puede tomar valores enteros positivos, incluido el cero.

Sobre un semáforo sólo se pueden realizar 3 operaciones, las cuales se realizan de forma indivisible y por tanto no existen problemas de interfiliación. Estas operaciones son:

- ☞ *Inicialización:* asignarle un valor entero positivo a un semáforo. Sólo se puede realizar una asignación, lo que se hace al principio de la implementación.

SI hay procesos bloqueados ENTONCES desbloquear uno
SINO $S := S + 1$

- ☞ *Señalización:* comprueba si hay procesos bloqueados. Si los hubiera, desbloquea uno y en caso contrario incrementa el valor del semáforo.

SI $S = 0$ ENTONCES bloquear proceso
SINO $S = S - 1$

- ☞ *Wait:* si el valor del semáforo es cero, el proceso se bloquea. En caso contrario se decrementa en 1 el valor del semáforo.

Los semáforos se utilizan para sincronizar y resolver el problema de la exclusión mutua.

Propiedades:

1. Los semáforos implementan correctamente el problema de la exclusión mutua para cualquier número de procesos y para cualquier número de procesadores.
2. Puede ocurrir inanición a menos que se siga un orden FIFO en la cola de procesos bloqueados.
3. Un semáforo que se utiliza para resolver exclusión mutua está sujeto a interbloqueos.
4. La comunicación de procesos queda bajo responsabilidad de los programas. Los semáforos sólo hacen su función.

Ejemplo:

P1	P2
a) Temp1 = y + 3 · x; Signal (s);	b) Temp2 = v + t; Wait (s) c) a = Temp1 + Temp2 ; Write (a) ;

Si S = 0 se bloquea el proceso.

Signal (s) b Indica que se ha ejecutado la instrucción anterior.

Tipo de semáforos:

1. *Generales o contadores:* son los vistos hasta el momento. Nos sirven como contadores. (Asignándole a S un valor de 10 tendremos un bucle que se repite 10 veces).
2. *Binarios:* toman valores booleanos, 0 ó 1.

Implementación de semáforos:

Necesitamos de una estructura de datos enteros donde se almacene el número del semáforo y los punteros a los procesos en cada momento.

Contador entero	Puntero a cola de bloqueados	Puntero a cod. de g. cola
--------------------	------------------------------------	---------------------------------

Para almacenar los procesos bloqueados necesitamos un puntero a la cola de bloqueados y cada semáforo poseerá una cola de bloqueados.

Una vez ejecutado **Wait (s)** se incorporaría el proceso actual en la cola de bloqueados. Para gestionar dicha cola se utilizará una rutina de tratamiento, con un puntero a la rutina.

Las rutinas de tratamiento pueden ser la siguientes:

- ↳ *FIFO:* facilidad de implementación y manejo de la cola.
- ↳ *PRIORIDADES:* en caso de existir varios procesos con distinta prioridad que compartan información.
- ↳ *LIFO:* no se suele utilizar.

Otra implementación de semáforos trata de conseguir que las operaciones **Wait** y **Signal** sean atómicas, esto es, las operaciones sobre semáforos sean indivisibles. Esta atomicidad se puede conseguir de dos maneras:

1. *Sistemas monoprocesadores*: se consigue activando y desactivando las interrupciones.
2. *Sistemas multiprocesadores*: mediante las instrucciones **Test & set** ó **Test & set block** si el sistema las permite. En caso contrario utilizaremos algoritmos software con espera ocupada. También se puede conseguir ésta mediante la centralización en un procesador de un planificador que gestione sus semáforos.

Problemas:

1. Productor / consumidor con buffer ilimitado.

Se explicará este problema utilizando un ejemplo que cuenta con dos procesos, uno productor y otro consumidor que cumplen:

- i. Un proceso, Productor, que continuamente introduce elementos en un buffer BUFF.
- ii. Un proceso, Consumidor, que continuamente saca elementos de un buffer BUFF.

PRODUCTOR		CONSUMIDOR
While true DO Producir (D); Wait (mutex); BUFF[i] := D; Signal (mutex); Signal (vacio); i := i+1; ... END {While}		While true DO Wait (vacio); Wait (mutex); R := BUFF[j]; Signal (mutex); j := j+1 ... END {While}

Inicialmente:

- ⌘ BUFF está vacío.
- ⌘ i,j := 1
- ⌘ mutex := 1 (controla el problema del acceso mutuo a BUFF).
- ⌘ vacío := 0 (controla que el proceso consumidor no intente leer de BUFF cuando está vacío).

Se ha de cumplir las siguientes condiciones para los dos procesos anteriores:

- a) El proceso consumidor no puede coger datos de BUFF (buffer) si este está vacío. Esto se controla con la variable vacío.
- b) Ambos procesos comparten datos comunes, presentando el problema de exclusión mutua, esto es, ambos no pueden acceder a la vez a datos compartidos (se soluciona con mutex).

El proceso productor indicará que el buffer se está llenando mediante signal.

El interbloqueo (deadlock) ocurre cuando se ejecutan procesos concurrentes. Es una situación en la cual un conjunto de procesos se quedan bloqueados esperando que ocurra algún suceso que solamente un proceso de dicho conjunto puede provocar. Este interbloqueo se produce por el intento de acceder a datos compartidos.

2. Productor / consumidor con buffer limitado.

Suponemos que BUFF es un buffer con capacidad limitada a N elementos (implementado como una lista circular).

PRODUCTOR		CONSUMIDOR
While true DO Producir (D); Wait (espacio); Wait (mutex); BUFF[i] := D; Signal (mutex); Signal (vacío); i := (i+1) mod N; ... END {While}		While true DO Wait (vacío); Wait (mutex); R := BUFF[j]; Signal (mutex); Signal (espacio); j := (j+1) mod N; ... END {While}

Inicialmente:

- ✕ BUFF está vacío
- ✕ i,j := 1
- ✕ mutex := 1
- ✕ vacío := 0
- ✕ espacio := N (controla que el proceso PRODUCTOR no introduzca más elementos en BUFF si éste no está vacío).

El PRODUCTOR no debe introducir más datos en el buffer si este está lleno, pues se machacarían datos al ser el espacio limitado.

Cuando se produce un elemento hay que comprobar si existe o no espacio para ello.

Hay que tomar precauciones con las operaciones **Wait** dentro de la sección crítica, pues puede producir un problema de interbloqueo.

Limitaciones de los semáforos:

1. Este tipo de semáforos presupone la existencia de una memoria común. No es una solución viable la utilización de semáforos para los sistemas distribuidos (tiempo de comunicación elevados). Por esto se utilizan en sistemas centralizados.
2. La comunicación entre procesos concurrentes queda bajo la responsabilidad del programa de aplicación. Esto es, de ello no se encargan los semáforos.
3. No asegura la buena utilización de **Wait** y **Signal** por lo que se dice que los semáforos son un mecanismo no estructurado.
4. La implementación de semáforos produce código difícil de verificar.

4.- Mensajes.

Definición:

Son un mecanismo sencillo en su implementación que resuelve algunas limitaciones de los semáforos. Se utilizan por ello tanto en sistemas centralizados como en sistemas distribuidos.

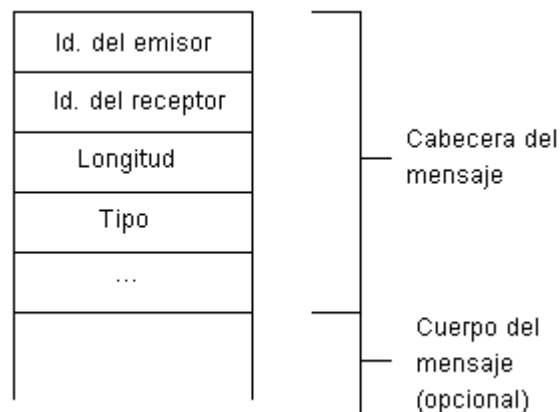
Se utilizan para resolver problemas como sincronización, comunicación y exclusión mutua.

Otra definición de mensajes: una colección de información intercambiada entre un proceso emisor y un proceso receptor.

Formato:

Cabecera: formato fijo.

Cuerpo: opcional y de longitud variable.

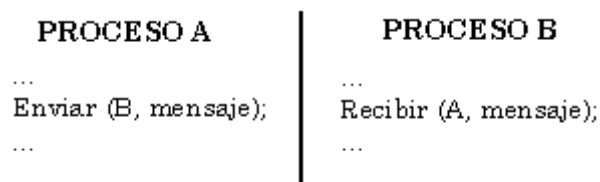


Operaciones:

- ♣ Enviar o SEND.
- ♣ Recibir o RECEIVE.

Implementación de mensajes:

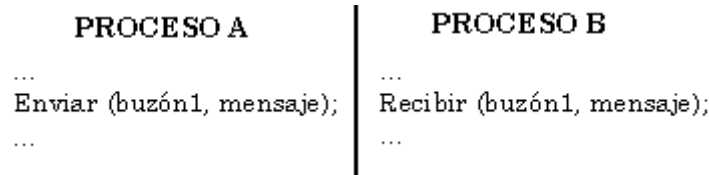
1. Designación del emisor / receptor:
 - a. *Directa*: cada emisor designa al receptor específico y viceversa. El receptor debe conocer el nombre del que envía el mensaje.



Ventajas: no existen fallos de identificador, de receptor o de emisor.

Desventajas: ha de conocer en todo momento todos sus posibles clientes.

- b. *Indirecta*: Los mensajes son enviados y recibidos a través de depósitos especiales llamados **buzones**. El emisor manda los mensajes a un buzón, que es un lugar que ha sido creado con anterioridad, tomando el receptor los mensajes de dicho buzón.



El propio sistema ha de gestionar a los buzones, proporcionando instrucciones para mejorar estos.

Es necesario disponer de servicios adicionales para el mantenimiento de los buzones:

- ☞ Crear_buzón.
- ☞ Borrar_buzón.

Con este tipo de mensajes de designación indirecta, la comunicación podrá ser:

- ☐ 1 a 1.
- ☐ 1 a muchos.
- ☐ Muchos a 1.
- ☐ Muchos a muchos.

Un buzón es una cola donde se almacenan los mensajes.

2. Intercambio síncrono y asíncrono de mensajes

a. *Síncrono*:

La comunicación se realiza sólo cuando ambos procesos, emisor y receptor, están separados.

Sólo puede existir un mensaje pendiente como máximo en cada momento por cada par emisor – receptor.

La operaciones **Set** y **Receive** van a ser bloqueantes. Si un proceso realiza una operación enviar y otro proceso una operación recibir sobre el mismo buzón, el primer proceso va a estar esperando hasta que el segundo reciba y viceversa.

Ventajas:

- ☞ Fácil implementación.
- ☞ Seguridad en la recepción del mensaje.

Inconvenientes: sincronización forzosa.

b. *Asíncrono*:

El proceso emisor no se bloquea, aunque el proceso receptor no esté preparado (no haya ejecutado una operación recibir).

El sistema operativo acepta y almacena temporalmente los mensajes pendientes hasta que se reciban (recargo adicional para el sistema).

Aumenta la concurrencia.

Por un mal uso, un proceso puede agotar la capacidad de almacenamiento intermedio (temporal) del sistema. Como el S.O. no posee espacio de almacenamiento, los procesos quedarían bloqueados.

Problema común: aplazamiento indefinido. Se produce por las siguientes causas:

- ♣ Se envía un mensaje y nadie lo recibe. Se bloquea en síncrono.
- ♣ Un mensaje nunca se envía. Se bloquea en síncrono y en asíncrono.

Esto se soluciona implementando la operación recibir con otro parámetro, siguiendo la sintaxis:

Recibir (buzón, mensaje, límite de tiempo).

El límite de tiempo es el tiempo que va a esperar el proceso como máximo para recibir un mensaje (proceso receptor). Si el proceso receptor sobrepasa ese límite de tiempo esperando un mensaje que no llega, dicho proceso se desbloquea y manda a su vez un mensaje informando que se ha sobrepasado el límite de tiempo.

PRODUCTOR CONSUMIDOR CON BUFFER LIMITADO: MENSAJES

PRODUCTOR	CONSUMIDOR
While true DO	While true DO
RECIBIR (puede_producir, psmj);	RECIBIR (puede_consumir, csmj);
psmj := Producir (D);	Consumir (cmsj);
ENVIAR (puede_consumir, psmj);	ENVIAR (puede_producir, csmj);
...	...
END {While}	END {While}

Inicialmente:

- ⌘ Crear_buzón (puede_producir);
- ⌘ Crear_buzón (puede_consumir);
- ⌘ FOR i=1 TO capacidad_buzón DO ENVIAR (puede_producir, null);
- ⌘ Las variables psmj y cmsj son locales a los procesos productor y consumidor respectivamente.

RECIBIR i WAIT
ENVIAR i SIGNAL

c) *Copia.*

La transferencia del contenido del mensaje del espacio de dirección del emisor al del receptor se puede hacer mediante los siguientes métodos:

1. *Por valor:*
 - ⌘ Más seguridad
 - ⌘ Si el receptor manipula la copia del mensaje, para el emisor es invisible porque no comparte el mismo espacio de memoria
 - ⌘ Se copia todo el mensaje al espacio de direcciones del receptor.
2. *Por referencia:*
 - ⌘ Más eficiencia
 - ⌘ Se pasa un puntero al mensaje entre los dos procesos.
 - ⌘ Vamos a ganar en recursos. Se pasa al procesos receptor el mismo espacio de direccionamiento que el emisor, con el inconveniente de que si un proceso cambia algo, dicho espacio va a afectar al otro.
 - ⌘ Una ventaja será la velocidad, que ahora es mayor. Es necesario introducir recursos para compartir memoria.
3. *Estrategia híbrida:*

Se hace una sola copia, pero sólo en el caso de escritura se realiza una copia real.

En sistemas centralizados se pueden utilizar cualquiera de las tres, pero en sistemas que no trabajan con memoria común no puede hacerse por referencia.

Longitud de los mensajes:

El problema se presenta no en los pasos por referencia, sino en los pasos por valor. Se necesitan mecanismos de secuenciamiento pues los mensajes van a ser troceados y enviados. Hay varios mecanismos para controlar dicho envío:

- ↳ *Fijo*: se trocea el mensaje en submensajes y luego se envía de forma ordenada.
- ↳ *Variable*: el S.O. va a tener mecanismos adicionales de manejo de memoria para la creación de buffer para que se ajuste a los mensajes enviados. Por tanto no hay que trocear los mensajes, pero va a ser más costoso.

Tema 4: Gestión de memoria

1. **Introducción.**
2. **Sistema monoprogramado.**
3. **Sistema multiprogramado:**
 - 3.1. **Esquema de particiones fijas**
 - 3.2. **Particiones variables.**
 - 3.3. **Protección.**
4. **Recubrimientos (overlay).**
5. **Intercambios (swapping).**

1. Introducción.

Para que un proceso se pueda ejecutar necesita memoria. Un módulo muy importante del S.O. es el que gestiona la memoria. La gestión de memoria mediante un gestor de la misma, favorece el rendimiento del sistema. El principal objetivo es asignar una cantidad de memoria física a los distintos procesos que la soliciten. La memoria es un recurso vital.

Dentro de la gestión de memoria vamos a ver 3 aspectos fundamentales:

- 1.1. *Organización:* Hay que saber si el sistema va a ejecutar un proceso de usuario o varios procesos de usuario. Si la memoria puede contener varios procesos hay que saber como se va a dividir.
- 1.2. *Manejo de memoria:* o como se lleva la cuenta del espacio de memoria ocupado y del espacio que queda libre. Existen distintas estrategias:
 - 1.2.1. *Estrategia de ubicación:* o asignación de memoria, cuando un proceso llega al sistema donde se va a ubicar.
 - 1.2.2. *Estrategia de sustitución:* en caso de falta de memoria, que datos y programas vamos a sacar de memoria para meter otros.
 - 1.2.3. *Estrategia de captura:* como recuperar parte de un programa y sus datos en caso de que los requiera para su ejecución. Podemos distinguir 2 casos:
 - 1.2.3.1. *Por demanda:* sólo cuando el proceso realmente necesite cargar esa parte del espacio de direcciones para su ejecución, se cargará.
 - 1.2.3.2. *Por anticipación:* el sistema previamente va a cargar las partes del proceso antes de que las necesite para su ejecución.

- 1.3. *Protección:* tenemos que proteger la zona de memoria reservada al S.O. de los accesos indebidos por parte de los procesos de usuario así como los espacios de direcciones de los procesos del resto de los procesos de usuario. El S.O. también se encarga de la compartimentación del espacio de los distintos procesos.

El rendimiento del sistema se ve afectado por la eficacia del módulo del gestor de memoria. Vamos a medir la eficacia centrándonos en dos aspectos fundamentales:

- 1.3.1. *Memoria desaprovechada:* o fragmentación de la memoria principal no utilizada, que puede provocar que el sistema sea incapaz de asignar memoria a los procesos que la solicitan.
- 1.3.2. *Tiempo dedicado por el sistema a la asignación de memoria:* si se usan algoritmos demasiado complejos, el módulo del gestor de memoria hace que el rendimiento del sistema sea más pobre.

Un programa escrito en un lenguaje fuente no es un programa ejecutable hasta que no se traduce en código máquina y no será un proceso hasta que no se le asigne memoria.

Tenemos un programa con una zona de código y otra de datos. Cuando cargamos el programa en memoria creamos un PCB, el código, datos y la pila. A éste conjunto lo llamamos **imagen del proceso**. Un proceso no será ejecutable hasta que no tenga una imagen.

En un sistema multiprogramado, el usuario no conoce donde se va a cargar el programa, depende del espacio libre de memoria principal. Se crean programas reubicables.

Concepto de reubicación

Es la capacidad de cargar un programa en un lugar arbitrario de la memoria en oposición de un conjunto de direcciones fijada en tiempo de traducción.

¿Cómo se carga un programa en memoria principal?. Una forma utilizada es la carga absoluta. Creamos un espacio de direcciones del programa que será físico, es decir, direcciones reales en memoria. Esto no es eficiente porque no podemos saber que posibles direcciones de memoria están libres.

Lo que hacemos es que cada vez que se compila el programa, se crea un espacio de direcciones lógico. Cada vez que carguemos el programa ejecutable tiene que acceder a posiciones físicas de memoria. Los cargadores son relativos ya que trabajan con espacio de direcciones lógico.

Dependiendo de cómo se realice la traducción de direcciones lógicas a físicas, tendremos 2 tipos de reubicación:

- ♣ *Estática*: se hace la traducción en el momento de la carga, cuando el gestor de memoria encuentra un sitio se carga el programa.

$$\text{Dirección física} = \text{dirección lógica} + \text{dirección de comienzo}$$

- ♣ *Dinámica*: se realiza la traducción de direcciones en tiempo de ejecución. El programa se carga en memoria sin traducir sus direcciones. Se van traduciendo a direcciones físicas según se va ejecutando mediante un soporte hardware.

$$\text{Dirección física} = \text{dirección lógica} + \text{contenido del registro base}$$

En resumen, la ligadura de direcciones lógicas a direcciones físicas se puede hacer en 3 momentos:

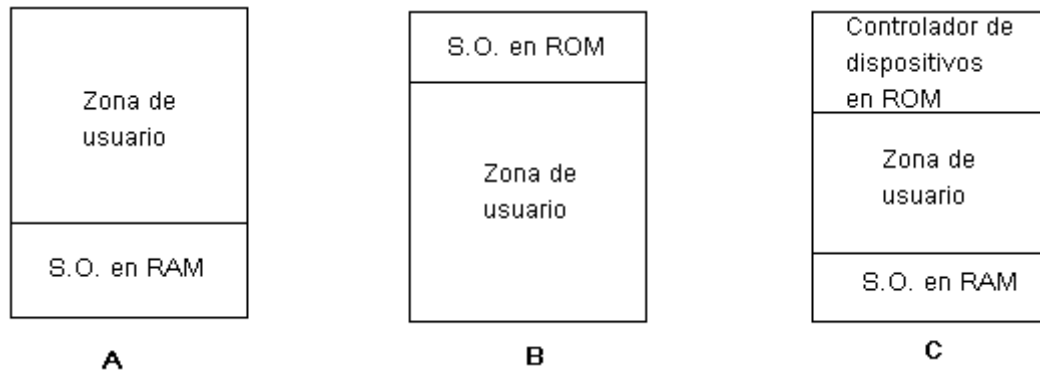
- ☞ *Tiempo de compilación* b cargador absoluto.
- ☞ *Tiempo de carga* b reubicación estática.
- ☞ *Tiempo de ejecución* b reubicación dinámica.

2. Sistema monoprogramado.

En estos sistemas se permite la ejecución de un solo programa en memoria principal. Los primeros S.O. carecían de gestor de memoria y era el propio usuario el que proporcionaba los controladores y administraba la memoria. Como sólo podía existir un usuario a la vez, no había problemas de protección.

Cuando se crearon los S.O.'s la organización de la memoria se basó en dividir la memoria principal en 2 partes, una para que residiera el S.O. y otra para procesos de usuarios (procesos transitorios). Se pensó poner el S.O. en ROM pero el problema es que entonces no sería modificable.

Otra de las opciones manejadas fue la de poner el controlador de dispositivos en ROM, el S.O. en RAM (zona baja) y la zona alta para procedimientos de usuario, tal como muestran las figuras:



En el esquema **A** sólo tenemos que preocuparnos por la **protección** del sistema operativo y los procesos de usuario.

Protección de memoria

En un registro almacenamos la posición de memoria donde se encuentra la frontera del S.O. y la zona de usuario. A dicho registro se le llama *valla* o *registro límite*. Cuando se genera una dirección, la C.P.U. compara la dirección a la que quiere acceder con el valor del registro límite. Si la dirección generada es mayor o igual que el registro límite, entonces es correcta. En caso contrario, se produce un error al intentar acceder a una posición de memoria del S.O..

Esta comprobación se hace mediante hardware ya que sería muy lento el hacerla mediante software.

Para este esquema de protección, el procesador tiene que funcionar en modo supervisor y usuario para poder generar interrupciones.

Es sencillo de utilizar, necesitamos un comparador, pero el rendimiento del sistema no se puede optimizar si sólo podemos tener un proceso en memoria principal

3. Sistema multiprogramado

Vamos a ver esquemas de gestión de memoria, considerando que la asignación de memoria va a ser contigua. No podemos dividir un proceso en 'trozos' y poner cada 'trozo' en una zona de memoria distinta.

3.1. Esquema de particiones fijas

Tenemos que repartir la memoria en varios procesos de usuario. Este esquema está basado en la asignación contigua de memoria cuando se carga el espacio de un proceso.

Consiste en dividir la memoria principal en un número de partes, posiblemente de tamaños distintos (aunque podrían ser del mismo tamaño). Ocurre cuando se inicializa el sistema, el tamaño es constante, no puede variar. Cuando se crea el proceso se carga en una de esas particiones.

Existe fragmentación interna, es decir, el proceso ocupa menos espacio que el tamaño de la partición asignada. El resto del espacio se desperdicia.

No podemos asignar 2 particiones a un mismo proceso de usuario. Cada partición puede albergar a un solo proceso aunque sobre memoria.

Decidiremos el número y el tamaño de particiones y el grado de multiprogramación (número máximo de procesos que puede haber en memoria principal).

El grado de multiprogramación depende del tamaño más típico de los procesos que se ejecutan en el sistema.

La división depende de:

- ✧ La cantidad de memoria principal disponible.
- ✧ El grado de multiprogramación deseado.
- ✧ Los tamaños típicos de los procesos más frecuentemente ejecutados.

Pretendemos obtener el máximo rendimiento del sistema-

Este esquema es válido para sistemas estáticos donde se puede prever el tamaño de los procesos que se van a ejecutar.

Para saber el número de particiones asignadas y las que están libres, el sistema utiliza una tabla donde se guarda el inicio de la partición, el estado (libre u ocupada) y por que proceso está ocupada.

<i>Tamaño</i>	<i>Dirección</i>	<i>Estado</i>	<i>Proceso</i>

Otra parte de la gestión de memoria es los métodos de organización de memoria a los distintos procesos.

Podemos tener una cola por cada partición aunque la solución más óptima será tener una única cola para los procesos ya que podemos elegir la partición adecuada para este.

Así pues caben estas dos maneras de asignación de los procesos a las particiones:

- ⌘ Podemos tener una cola de procesos por partición que controla la asignación de particiones a procesos del sistema. Cuando el S.O. recibe el proceso y prevé la cantidad de memoria que requiere, lo asignamos a una determinada cola de una partición.
Existe un desperdicio de memoria debido a las particiones que tienen las colas vacías (procesos poco frecuentes que requieren mucha memoria).
- ⌘ Podemos tener una única cola para todas las particiones. La existencia de particiones muy grandes o muy pequeñas también pueden provocar desperdicio de memoria.
El sistema genera una tabla de tamaño fijo donde tenemos una entrada por cada partición e informa sobre su estado.

Independientemente de la asignación de particiones a procesos, éste esquema de memoria tiene un problema de desaprovechamiento que llamamos **fragmentación interna**-

Una partición se le asigna completa a un proceso. Si este requiere menos memoria, la diferencia se desperdicia.

Podemos utilizar estrategias que minimicen la fragmentación interna

Se pueden utilizar 2 algoritmos para asignación de particiones:

- ⌘ *Primer ajuste (más rápido, menos complejo)*: buscamos en la memoria la 1ª partición libre donde quepa el proceso (el tamaño de la partición es mayor o igual que la memoria requerida por el proceso).
- ⌘ *Mejor ajuste (más complejo, más lento)*: buscamos la partición que mejor se adapte a los requisitos de memoria. Proporciona una fragmentación interna menor que el primer ajuste.

3.2. Particiones variables.

En las particiones fijas se limitaba el número de procesos en memoria se, además algunas particiones se desperdiciaban a causa de que el proceso era menor que la partición.

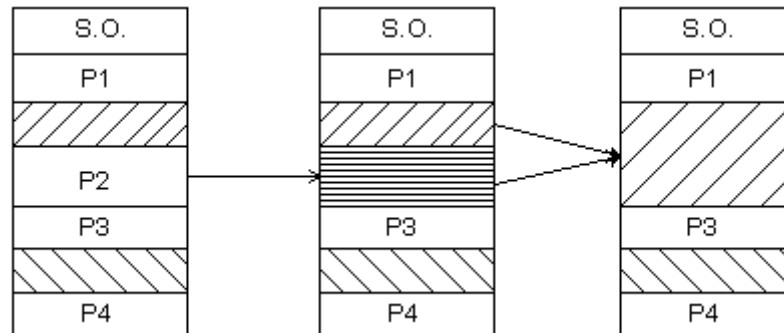
Esto se arregla con una **partición dinámica**, de manera que las particiones no se hacen de antemano sino que conforme van llegando los procesos, el sistema va realizando las particiones.

No hay desperdicio, la partición es exactamente igual al tamaño. El problema es que cuando acaban los procesos, estos dejan huecos en el almacenamiento principal. Estos huecos pueden ser usados por otros procedimientos de igual o menor tamaño que el hueco dejado, pero aunque esto suceda, los huecos restantes van haciéndose más pequeños hasta el punto que no podrán ser asignados. Es lo que se conoce como **fragmentación externa**.

Algunas soluciones:

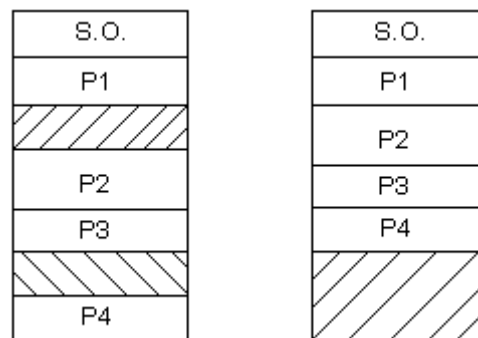
☞ *Fusión de huecos adyacentes.*

Cuando termina un proceso podemos comprobar si el almacenamiento que libera tiene límites con otros agujeros y fusionarlos, formando un hueco de mayor tamaño.



☞ *Compactación de la memoria.*

Consiste en trasladar las áreas ocupadas de memoria a uno de los extremos de la memoria principal, dejando a su vez un solo agujero grande de memoria libre.



Algunos problemas de la compactación son:

- ☞ Pérdida de tiempo: durante la compactación el sistema no trabaja.
- ☞ La compactación sólo se puede realizar en el caso en que los programas sean reubicables dinámicamente.
- ☞ Si aparecen y desaparecen frecuentemente procesos, es necesaria más compactación.

Gestión de memoria

Para ver como se guarda la información y como se realiza el manejo de memoria calculando que parte está libre y cual ocupada, vamos a ver 2 procesos:

☞ *Mapa de bits:* consiste en dividir la memoria en unidades de asignación que pueden tener un tamaño desde una palabra a varios Kbytes. Cada unidad de asignación se representa por un bit, '1' si está ocupada y '0' si está libre. Representa toda la memoria del sistema. El tamaño es 'n' siendo 'n' el número de asignaciones.

Un problema es el tamaño de las unidades de asignación ya que:

- ♣ Si son pequeñas, el mapa de bits es muy grande y se tarda mucho en recorrerlo, actualizarlo y encontrar unidades libres.
- ♣ Si son grandes, el mapa de bits es muy pequeño y se desperdicia mucho espacio porque existe fragmentación interna.

- ↳ *Listas enlazadas:* cada nodo representa el espacio ocupado o memoria libre. Dependiendo de cómo implementemos la lista, obtendremos más eficacia a la hora de buscar huecos libres. Una de esas implementaciones es mediante listas doblemente enlazadas. Otra teniendo una lista con los huecos libres y otra con los procesos (si buscamos huecos libres solo tenemos que ir a la lista adecuada.

Algoritmos de asignación de memoria

Se utilizan para determinar, en la memoria principal, el lugar donde serán colocados los procesos y datos que van llegando. Los 3 más utilizados son:

- ✱ *El primer ajuste:* al llegar un proceso nuevo, recorre la memoria desde el principio hasta encontrar el primer hueco donde quepa. Toma con rapidez la decisión del emplazamiento.
- ✱ *El mejor ajuste:* recorre toda la memoria hasta encontrar el hueco en el que mejor se adapte y deje el menor espacio sin usar. Es el algoritmo que mayor número de huecos pequeños deja en memoria.
- ✱ *El peor ajuste:* coloca el proceso en el agujero más grande posible. Así, una vez colocado sigue quedando un hueco grande para albergar un nuevo proceso.

3.3. Protección.

- ⚠ *Reubicación estática:* cuando la C.P.U. genera una dirección, comprueba que esa dirección no sobrepase el límite superior de la partición que está ocupando el proceso. Cuando el proceso sobrepasa la cantidad de memoria que se le ha asignado, el sistema da un error.

$\text{Registro base} < \text{Proceso} < \text{Registro límite}$

- ⚠ *Reubicación dinámica:* El registro límite va a contener el número de direcciones total del proceso. El registro base va a contener la primera dirección a partir de la cual está cargando el proceso en memoria principal. Cuando la C.P.U. quiere acceder a memoria se compara la dirección lógica con el contenido del registro. Los registros forman parte del entorno volátil, por tanto están guardados en el PCB.

4. Recubrimientos (overlay).

Esta técnica aparece porque los datos de un proceso también están cargados en memoria principal para que el proceso se pueda ejecutar. El problema es que puede que no exista bastante memoria principal para albergar todos los datos de un proceso. El tamaño de los procesos está limitado por la cantidad de memoria principal, pero es posible ejecutar procesos más grandes por medio de los **recubrimientos** u **overlays**.

Esta técnica consiste en que cuando no todo el programa tiene que estar en memoria principal ejecutándose, metemos en memoria principal sólo aquellos datos o instrucciones que puedan ser utilizados en un momento dado.

Este método ofrece al programador una manera de extender el límite de memoria principal.

Dividimos el proceso en tres partes:

- 1.- Inicialización.
- 2.- Procesamiento.
- 3.- Salida.

Hay una torsión de código y datos de usuario que deben permanecer en memoria principal durante toda la ejecución. El resto de las partes se irán metiendo en memoria principal según su necesidad:

- 1.- Carga inicialización y ejecuta.
- 2.- Carga procesamiento y ejecuta.
- 3.- Carga salida y ejecuta.

Un problema es que la ejecución puede ser muy lenta ya que tiene que hacer continuas llamadas a overlays y si éstos no están bien diseñados, el sistema se satura.

El S.O. se encarga de las E/S (cargar y descargar overlays en memoria principal). La ejecución del programa es más lenta que si el proceso completo se encontrara en memoria principal.

En un sistema con particiones fijas o variables, un proceso puede ser más grande que la partición, siempre y cuando en la partición haya espacio suficiente para albergar los datos permanentes y los overlays.

Este método no se suele utilizar porque existen métodos más modernos para solucionar estos problemas (gestión de memoria virtual).

5. Intercambios (swapping).

Es otra técnica para resolver el problema de que existan en memoria principal procesos bloqueados que no pueden continuar su ejecución y que llegue un proceso nuevo y no se pueda ejecutar porque no hay espacio en memoria principal.

El intercambio se encarga de expropiar la memoria a procesos que se encuentran en memoria principal y de asignarla a procesos que se encuentran en memoria secundaria.

Se puede utilizar en particiones fijas, variables o sistemas monoprogramados al igual que el recubrimiento u overlay.

La técnica swapping no requiere que el proceso permanezca en memoria principal hasta su finalización. Un proceso ocupa el almacenamiento principal de inmediato, se ejecuta hasta que ya no puede continuar más y cede la memoria y la C.P.U. a otro proceso. La memoria principal se dedica a un proceso durante un breve periodo, ese trabajo es 'retirado' y entra el nuevo.

Un proceso puede ser intercambiado varias veces antes de llegar a su término.

El proceso 'retirado' de memoria principal es almacenado en un dispositivo de almacenamiento secundario rápido, donde se van a almacenar las imágenes de los procesos que se van a retirar temporalmente de memoria principal.

Si usamos reubicación estática el proceso 'retirado' deberá continuar (al entrar en memoria principal) en la misma posición de memoria de donde se sacó.

Al paso de memoria principal a disco se le llama **swapped out** y al paso de disco a memoria principal se le llama **swapped in**.

El **proceso intercambiador** o **swapper** se encarga de realizar el intercambio entre procesos. Tiene 3 funciones:

1. Seleccionar procesos para retirarlos de memoria principal. Hay que salvar la imagen del proceso retirado en un dispositivo de almacenamiento secundario que sea rápido. Los procesos seleccionados para retirar de memoria principal son:
 - ♣ Los procesos bloqueados.
 - ♣ Los procesos con baja prioridad.
2. Seleccionar procesos para incorporarlos a memoria principal. Es la función del planificador a corto plazo. Los procesos que se seleccionan para incorporación a memoria principal son aquellos que están en estado ejecutable pero residentes en memoria secundaria. Los criterios para la selección son:

- ♣ Por la edad (cuanto tiempo ha pasado en memoria secundaria).
- ♣ Por prioridades.
- ♣ Si ha pasado un tiempo límite en memoria secundaria (2 seg. típicamente). Para evitar continuas operaciones de E/S el tiempo límite se basa en función del algoritmo de procesamiento.

3. Gestionar y asignar el espacio de intercambio.

El intercambio es útil para implementar la multiprogramación en sistemas monoprogramados. La zona de procesos transitorios sólo puede albergar un proceso. Cuando éste se bloquea por una operación E/S o cualquier otro problema, se desaloja el proceso guardando la imagen en memoria secundaria y se carga una nueva imagen de otro procesos que se encuentra en memoria secundaria.

La zona de intercambios puede ser de 2 formas:

- a. *Intercambio dinámico*: se crea un espacio global en disco para todas las imágenes de los procesos que puedan ser intercambiadas en disco.
La desventaja es que el tamaño del archivo limita el número de procesos del sistema.
La creación de este archivo se realiza durante la inicialización por lo que no podemos tener más procesos que imágenes haya en el archivo.
 - ✖ Si el archivo es muy grande limita el tamaño en disco usado por archivos de usuario.
 - ✖ Si es muy pequeño tenemos poco espacio de swapping.
- b. *Intercambio estático*: cada vez que se crea un proceso se crea un espacio en el disco que va a albergar la imagen. Creamos un archivo por proceso y éste va a contener la imagen. El proceso ha de ser intercambiable.
Ventajas:
 - ✖ No limita el número de procesos mientras podamos crear archivos.
 - ✖ Elimina el problema del tamaño del archivo.
Desventajas:
 - ✖ Requiere más espacio en disco por lo que también desperdicia más.
 - ✖ Los accesos son más lentos.

En los primeros sistemas compartidos, se tenía la zona de usuario donde sólo podía residir un proceso y había que tener en disco el resto de procesos. Cuando el proceso finalizaba se guardaba la imagen de ese proceso en disco y se cargaba otro.

Existen procesos en un S.O. que no deben ser intercambiados. El S.O. debe proporcionar mecanismos para indicar si el proceso debe ser intercambiado o no. Los mecanismos están restringidos a ciertos procesos o a ciertos usuarios. A los procesos no intercambiables no se les crea área de swapped.

Tema 5: Organización de la memoria virtual

1. Introducción.

La memoria virtual se basa en la idea de que el tamaño requerido por un proceso sea mayor que el que se dispone, solucionando el problema de la necesidad de que el usuario realice las particiones.

En memoria virtual, el espacio de direcciones de memoria virtual de un proceso no ha de estar cargado en memoria principal para ejecutarse.

Los mecanismos que implementan la memoria virtual son la segmentación y la paginación. Para ello se requiere un almacenamiento del espacio virtual a dos niveles.

Básicamente la memoria virtual consiste en que disponemos de menos memoria que la requerida para un proceso. Esto se soluciona realizando particiones del proceso de un tamaño igual al que se dispone, cargando sólo en un instante una partición, quedando las restantes particiones en disco, de tal forma que si se desea acceder a una dirección de memoria del proceso que se encuentra en disco, se carga dicha partición de memoria.

El espacio de direccionamiento virtual de un proceso puede ser mayor o igual que el espacio de direccionamiento real.

La memoria virtual es útil para realizar un aumento de la multiprogramación, esto es, existencia de más procesos en el sistema, aumentando el rendimiento.

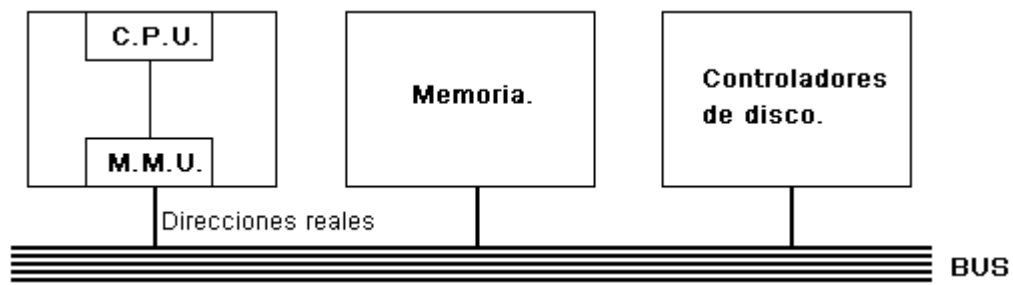
La memoria virtual requiere que se realice un almacenamiento a dos niveles:

- ☒ *Memoria real:* donde se almacena parte del proceso.
- ☒ *Memoria secundaria (disco):* donde se almacena todo el espacio requerido por el proceso. La imagen completa del espacio de direccionamiento virtual del proceso.

Para realizar el almacenamiento anterior, se necesitan básicamente dos cosas:

- ✧ Se debe saber que partes de la imagen del proceso están en memoria principal y que partes no lo están.
- ✧ Debe existir una política de movimiento entre memoria principal y disco, que sea efectiva (eficiente). El hecho de tener partes del proceso en memoria secundaria va a ralentizar la velocidad de tratamiento o la ejecución del programa.

Existe un mecanismo hardware de traducciones, encargado de traducir de forma simultánea las direcciones virtuales a reales. Esto implica una mayor velocidad que si se realizase mediante un mecanismo software. A dicho traductor hardware se le denomina **M.M.U. (Memory Management Unit -> Unidad de gestión de memoria)**. La C.P.U. genera direcciones virtuales, por tanto, entran en el **M.M.U.** que genera direcciones físicas que ya si van al bus de sistema.



El **M.M.U.** realiza las siguientes operaciones:

- a. Traduce las direcciones virtuales en direcciones reales.
- b. Detecta si la dirección obtenida está en memoria principal o no lo está.

- c. Si no está en memoria principal entonces produce una interrupción porque se encuentra en disco, encargándose el S.O. de cargar el nuevo bloque que estaba en disco, en memoria principal.
- d. Resuelve los procesos que se producen en el crecimiento de procesos, de tal forma que si un proceso crece aumenta la memoria dedicada a este.

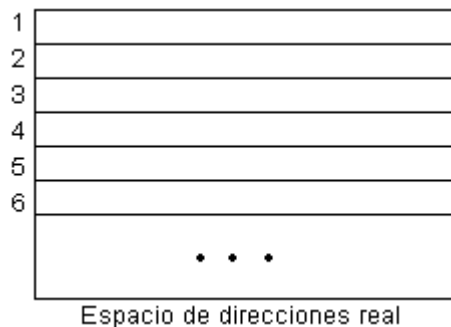
2. Organización de la memoria virtual:

2.1. Paginación.

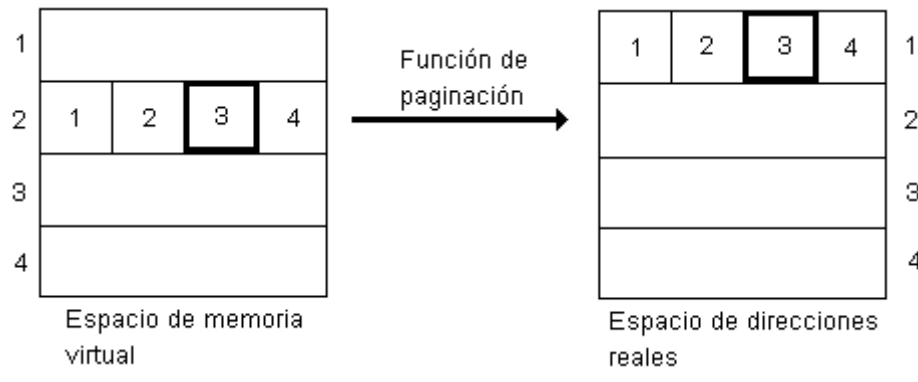
El mecanismo de paginación consiste en dividir la memoria virtual en bloques de tamaño fijo llamados páginas (esta división es totalmente transparente para el usuario).



La memoria real se divide en bloques del mismo tamaño que las páginas. A estos bloques se les denomina marcos de página.



La función de paginación es el mecanismo que se va a encargar de realizar una correspondencia entre los marcos de páginas y los números de páginas del direccionamiento virtual del proceso. De realizar esta función se va a encargar el **M.M.U.**



La dirección virtual se descompone en dos partes: el número de página y el desplazamiento dentro de la página.

El **M.M.U.** asocia números de página con números de marcos, siempre que la página está almacenada en memoria.



Para realizar la traducción de direcciones se va a necesitar mantener una tabla por proceso que va a mantener la información necesaria para esa traducción. Dicha tabla va a representar la información de un determinado espacio virtual, en la cual cada entrada va a representar la información referente a un proceso. A esta tabla se denomina **tabla de páginas** y su contenido es:

- ✕ *Número de marcos de página:* dirección real donde se almacena el proceso.
- ✕ *Indicador de presencia:* suele ser un bit que nos indica si la página está presente en memoria principal (bit a '1') o no (bit a '0').
- ✕ *Indicador de modificación:* se puede representar mediante un bit que indica si la página ha sido modificada después de su carga en memoria principal (bit a '1') o no (bit a '0').
- ✕ *Protección:* modo autorizado de acceso, secuencia de bits de protección. Podríamos utilizar un bit para la protección de lectura / escritura de tal forma que si está a '1' permite las anteriores operaciones y si está a '0' sólo permitirá la lectura. La protección de procesos adquiere importancia cuando se produce competencia entre páginas.

Para acceder a la tabla de páginas es necesario mantener en un registro de la C.P.U. la dirección base de esta tabla. Esta dirección también ha de estar almacenada en el descriptor de procesos.

Si al ejecutar la interrupción de falta de página no se encuentra un marco libre, este problema se solucionará de dos posibles modos:

1. El proceso estará esperando hasta que exista un marco libre en memoria principal.
2. Se intenta desplazar una de las páginas de memoria principal para hacer un hueco en este espacio de memoria y poder alojar la página deseada, esto se realiza mediante algoritmos de sustitución. Si no se puede desplazar la página el proceso queda bloqueado.

En caso de haber encontrado un marco libre, se carga la página en el marco que se había encontrado. Tras esto se cambia el número de marco de página por el número de marco que se había encontrado y con esto finalizamos.

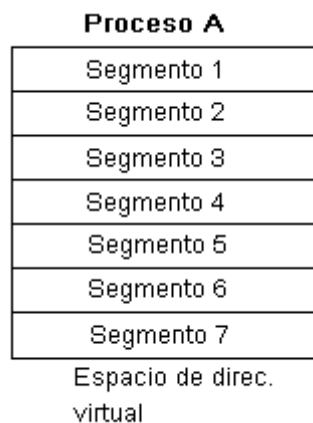
Problema de fragmentación interna

Al dividir el programa en páginas de tamaño fijo, si un programa ocupa menos espacio que el tamaño de una página, se estará desperdiciando el restante espacio de la página, no pudiendo ser éste asignado a otro proceso aunque cupiera (o cupiese) en dicho espacio.

Las ventajas de la fragmentación interna son la posibilidad de compartir programas (editores, compiladores,...) compartiendo páginas correspondientes al código entre los distintos programas, con el fin de no almacenar varias veces la misma información. Se permite compartir la parte de código no reentrante (no modificable), los datos de cada usuario serán distintos y no compartidos por tratarse de código no reentrante.

2.2. Segmentación.

La segmentación, al contrario que la paginación, se basa en la división del espacio de direccionamiento virtual en unidades de tamaño variable llamadas segmentos.



El tamaño de los segmentos es variable. Estos segmentos corresponden a unidades lógicas de un proceso. Esta división es realizada por el usuario (programador). Por ejemplo, se podría dividir un proceso en tres segmentos, segmento de pila, segmento de datos y segmento de código, donde el tamaño de cada segmento varía dependiendo de la unidad lógica que almacena. A esta división se denomina división en tiempo de ejecución.

El espacio de direcciones virtual en paginación es unidimensional, porque el usuario no tiene conocimiento de la división en páginas de dicho espacio.

Por el contrario, en segmentación se dice que el espacio de direccionamiento virtual es bidimensional porque al dividirse en segmentos, cada uno de estos comienza en la dirección cero y concluye en la última dirección del espacio, de tal forma que el programa cuando alude a un segmento, se ha de referenciar el número de segmento y el desplazamiento relativo dentro del segmento. Se ha de cumplir por esto que los elementos de un segmento se ha de almacenar en posiciones contiguas de memoria.

Se facilita la compartición entre procesos, pero se produce el problema de fragmentación externa, esto es, generación de huecos que no van a poder ser realojables. Para el alojamiento de segmentos es necesaria una asignación dinámica de memoria. Para ello utilizaremos algoritmos de particiones variables: *Mejor ajuste, primer ajuste y peor ajuste*.

La estructura de datos necesaria será una tabla de segmentos con una única entrada por cada segmento, donde se encuentran almacenadas la dirección, tamaño, etc. Se ha de almacenar la dirección de la tabla de procesos en el descriptor de procesos.

La información que se almacenará en dicha tabla será la siguiente:

- ⌘ *Dirección base del segmento (s)*: dirección de memoria a partir de la cual comienza el segmento si está cargado en memoria principal.
- ⌘ *Tamaño del segmento*: información necesaria para conocer si una dirección se encuentra fuera de las posibles direcciones de un proceso.
- ⌘ *Bits de protección*: bits que aluden a la protección de escritura, lectura, etc. con el fin de permitir estas o no cuando se comparten páginas.
- ⌘ *Indicador de presencia*: si el proceso está o no en memoria principal.
- ⌘ *Bit de modificación*: indica si se ha modificado el proceso, tras su carga en memoria principal.

Con el número de segmento y el desplazamiento, se accede a la entrada correspondiente en la tabla de procesos. Tras esto se mira si el proceso está en memoria principal o no. Si es así se comprueba si el desplazamiento es mayor que el tamaño del segmento, en caso afirmativo se produce una interrupción por acceso indebido a memoria. Y en caso de que el desplazamiento no sea mayor que el tamaño, mediante la concatenación de la dirección de comienzo del proceso en memoria principal con el desplazamiento, obtendremos la dirección real.

Si existe una interrupción por falta de segmento, el S.O. toma el control, realizando lo siguiente:

1. Bloquea el proceso.
2. Busca el segmento en disco, buscando la dirección de este en una tabla de ubicación en disco. La dirección de dicha tabla se almacena en el descriptor.
3. Una vez encontrado en disco el segmento, se busca una dirección de memoria contigua, donde se pueda almacenar el segmento. En el caso de no encontrar espacio de memoria, se puede optar por realizar un desplazamiento de otro proceso o realizar una espera hasta que haya espacio para almacenar este segmento.
4. Tras esto se carga el segmento.
5. Por último desbloqueamos el proceso y se obtiene la dirección real.

Se suelen utilizar mecanismos para solventar problemas mixtos. Uno de estos mecanismos es la **segmentación paginada**.

Tema 6: Gestión de la memoria virtual

1. Estrategias de gestión de memoria virtual con paginación.

Las distintas estrategias para gestionar una memoria virtual se van a basar en una serie de criterios:

- 1.1. Que cantidad de memoria principal se le asigna a cada proceso inicialmente. Este espacio de memoria puede ser:

- 1.1.1. *Constante*: se dice que el segmento de gestión se realiza mediante particiones fijas.

- 1.1.2. *Variable*: En este caso se habla de la utilización de una estrategia de partición de memoria mediante particiones variables.

- 1.2. Debemos tomar una decisión cuando haya que llevar una página de disco a memoria principal. Si no existe espacio libre en memoria principal entonces habría que sustituir una página. Para ello se ha de tomar una decisión con el fin de conocer que página se va a sustituir. Para ellos se utilizarán unos algoritmos de sustitución, los cuales dependiendo de la página que tomen se dividirán en:

- 1.2.1. *Locales*: la página a desplazar se elige del conjunto de páginas que se encuentran en el proceso que provocó la falta de página y que está en memoria principal.

M.P.	
a1	Se desplazarán las páginas del proceso que provocó la falta de página. En el ejemplo como este proceso es el 'a', sólo se podrán desplazar las páginas: a1, a2 y a3
a2	
a3	
b1	
b2	

- 1.2.2. *Globales*: la página a sustituir va a ser una que pertenezca a cualquier proceso que esté en memoria principal

M.P.	
a1	Se desplazará la página b1 y se introducirá la página a4
a2	
a3	
b1	
b2	

- 1.3. Cuando se va a pasar una página que está en disco a memoria principal, independientemente de que haya o no espacio, se siguen 2 variantes:
 - 1.3.1. *Paginación por demanda*: sólo se trae una página de disco a memoria principal cuando se produzca una falta de página, esto es, cuando se necesita traer la página a memoria principal realmente.
 - 1.3.2. *Anticipado*: se basa en que el S.O. puede determinar anticipadamente que páginas han de estar en memoria principal. Si se pudiera realizar dicha anticipación, se producirían menos faltas de página pero esto no ocurre así,

sino que existe una prepaginación, cargándose una serie de páginas en memoria principal.

Independientemente de los algoritmos de sustitución usados, existe una serie de criterios que siempre son válidos:

- ☛ *Páginas sucias*: aquellas que han sido modificadas tras su carga en memoria. Las *páginas limpias* serán aquellas que no han sufrido cambios tras su carga en memoria. Tendrán más probabilidad de ser elegidas como víctimas aquellas que no hayan sufrido ningún cambio, es decir, las *páginas limpias*.
- ☛ *Páginas compartidas*: si tengo una página compartida por varios procesos esto puede originar más falta de página. Por ello mientras exista otra posibilidad, estas no serán sustituidas.
- ☛ *Páginas especiales*: son páginas con una estructura especial, porque en un intervalo de tiempo, dicha página no debería ser sustituida. Una página que realiza una operación de E/S no debe sustituirse hasta que finalice la operación E/S.

¿Cuál es la influencia del tamaño de página?. Se encuentra implementando a nivel hardware, pero se puede realizar una estimación sobre el tamaño que debe tener una página.

☐ *Páginas muy pequeñas.*

- ✖ Las tablas de página se hacen muy grandes, el tiempo en gestionar ésta y la memoria ocupada será grande.
- ✖ Aumentan las transferencias disco – memoria principal, aumentando el tiempo en realizar operaciones de E/S.
- ✖ Disminuye la fragmentación interna.

☐ *Páginas muy grandes.*

- ✖ Aumenta la fragmentación interna.
- ✖ El proceso puede tener en memoria principal grandes cantidades de información que no van a ser utilizadas durante un largo periodo de tiempo.
- ✖ Los tamaños suelen oscilar entre 512 B., 1 KB, 2 KB y 4 KB.

2. Algoritmos de sustituciones locales.

Eligen como páginas víctimas a aquellas que se encuentran en el conjunto de páginas que se encuentran en el proceso que produjo la falta de página

2.1. Óptimo.

Este algoritmo es el que menos falta de página produce. Se desplazará aquella página que no será objeto de ninguna referencia posterior. Si no hay, se desplazará aquella que se referencie más tarde.

2.2. FIFO.

Es el que más falta de página produce. En este algoritmo la página a desplazar será aquella que se cargó con más anterioridad.

2.3. LRU.

Es, después del óptimo, el que menos faltas de página produce. La página a desplazar será aquella que fue objeto de la referencia más antigua, esto es, sustituye aquella página de las que se encuentran en memoria que se referenció en primer lugar. Para conseguir una buena gestión de memoria se intenta minimizar el número de faltas de página.

3. Comportamiento de los programas.

Los programas durante su ejecución no van a estar realizando referencias a memoria de forma aleatoria, sino que van a seguir un patrón de referencias a memoria no uniforme sino altamente local.

Las referencias a memoria no son aleatorias, sino que se referencian determinadas páginas en determinados espacios de tiempo. A esta propiedad de los procesos en tiempo de ejecución se denomina **localidad**, la cual es una propiedad empírica. Por ello, no la comparten todos los procesos, pero es altamente probable que la cumplan. En esta propiedad se pueden distinguir 2 tipos:

- ♣ *Temporal*: la localidad temporal nos dice que una posición de memoria referenciada recientemente tiene una alta probabilidad de ser referenciada en un futuro próximo. Ejemplo: si una rutina ha sido ejecutada, es muy probable que esa rutina vuelva a ser utilizada.
- ♣ *Espacial*: si cierta posición de memoria ha sido referenciada es muy probable que las posiciones de memoria adyacentes también lo sean. Por ejemplo en la ejecución secuencial de un programa se hace referencia a posiciones de memoria adyacentes (lectura del contenido de un array).

Va a existir un determinado intervalo de marcos de página (cantidad de memoria) asignada al proceso en el cual aumentan las faltas de página, en el caso de no poder mantener el conjunto de páginas referenciadas con frecuencia. A este conjunto se le denomina **conjunto de trabajo** o **working set**. Por lo tanto va a ser el conjunto de páginas referenciadas en un intervalo de tiempo (favorecidas durante un periodo de tiempo durante la ejecución de un proceso).

Si no podemos almacenar todas las páginas, se van a producir faltas de páginas que va a ir en aumento.

El no poder almacenar los programas va a venir dado por el amplio número de páginas a almacenar o por la sustitución de las páginas que se encuentran en éste.

El conjunto teórico en el que se basa este método se denomina **teoría del conjunto de modelo de trabajo** que dice: *“Para que un proceso pueda ejecutarse eficientemente, su conjunto de trabajo ha de estar en memoria principal. Por lo tanto, las estrategias de gestión de memoria virtual han de buscar este objetivo”*(la teoría del conjunto de trabajo dice que no se ha de desplazar ninguna página perteneciente al conjunto de trabajo de ningún proceso que se esté ejecutando o que esté en estado ejecutable).

Formalmente se puede decir que el conjunto de trabajo es para un determinado intervalo de tiempo t' y para un tamaño de ventana v el conjunto de páginas referenciadas entre los instantes de tiempo $(t-v, t)$ donde v es como dijimos el tamaño de ventana, esto es, el número de páginas que comparten el conjunto de trabajos (constante dada por un algoritmo).

Los conjuntos de trabajo son transitorios, esto es, van variando durante la ejecución de los programas. Esto presenta un grave problema que se soluciona mediante una serie de algoritmos que se basan en el pasado reciente de la ejecución de un programa.

4. Hiperpaginación.

Supongamos que en un momento dado la cola de ejecutables está vacía, con esto el planificador admitiría más procesos en el sistema, conllevando esto a un aumento de la multiprogramación. Se carga el proceso en memoria principal. Supongamos que los procesos se están ejecutando y se producen faltas de página. Mientras se produce esto, los procesos van a estar bloqueados. Como el sistema detecta que la cola de ejecutables está vacía, el sistema aceptará nuevamente procesos. Así sucesivamente, tras esto se irán produciendo nuevas faltas de página de tal forma que el sistema se encontrará solamente realizando tareas de paginación.

El fenómeno que se produce cuando el sistema está continuamente realizando actividades de paginación, afectando esto al rendimiento del sistema, se denomina **hiperpaginación**.

Cuando se produce la **hiperpaginación**, los trabajos que ejecuta el sistema van a disminuir y con esto el tiempo de respuesta de cada proceso también disminuirá.

Cuando se produce **hiperpaginación**, va a aumentar la falta de página, llegando un momento en el cual el trabajo del sistema es prácticamente nulo, esto es, cae el sistema.

La **hiperpaginación** se puede prevenir mediante:

- ✂ *La regulación de la carga:* es decir, mantener el grado de multiprogramación a un determinado número de procesos. Si se detecta que más del 50% del tiempo de C.P.U. se dedica a procesos, se disminuye el grado de multiprogramación estabilizando el número de procesos en memoria.
- ✂ *Algoritmos de sustitución globales:* miden de forma dinámica las páginas que un proceso necesita almacenar en memoria principal, para que no se produzcan faltas de página. Si necesita más páginas se asignan más marcos de página y si se necesitan menos se retiran marcos de página.